

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

**Xây dựng hệ thống tư vấn và tra cứu y tế kết hợp AI Agents và
RAG (Retrieval-Augmented-Generation)**

Vương Lê Bình Dương

duong.vlbbb226035@sis.hust.edu.vn

Program: Global ICT

Supervisor: Associate Professor Nguyễn Nhất Hải

Department: Information Technology

School: School of Information and Communications Technology

HANOI, 06/2026

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

**Xây dựng hệ thống tư vấn và tra cứu y tế kết hợp AI Agents và
RAG (Retrieval-Augmented-Generation)**

Vương Lê Bình Dương

duong.vlbbb226035@sis.hust.edu.vn

Program: Global ICT

Supervisor: Associate Professor Nguyễn Nhất Hải _____

Signature

Department: Information Technology

School: School of Information and Communications Technology

HANOI, 06/2026

ACKNOWLEDGMENTS

First and foremost, I would like to express my deepest gratitude to my supervisor, Associate Professor Nguyễn Nhất Hải, for his invaluable guidance, continuous support, and profound academic insights throughout the duration of this graduation thesis. His encouragement and dedication have been a constant source of motivation for my research and development.

I would also like to extend my sincere appreciation to the faculty of the School of Information and Communications Technology at Hanoi University of Science and Technology for providing a rigorous and stimulating academic environment.

Lastly, I am profoundly grateful to my family and friends for their unwavering love, patience, and support. Their belief in my potential has sustained me through the challenging phases of my studies and this research journey. This achievement would not have been possible without their presence in my life.

Vương Lê Bình Dương

ABSTRACT

In recent years, Large Language Models (LLMs) have created new opportunities for building medical question-answering and clinical decision-support systems. However, medical applications require stronger guarantees than ordinary conversational software. A clinical assistant must reduce hallucination, preserve privacy, trace its answers to retrieved evidence, and support practical workflows such as laboratory-report interpretation, medication lookup, and follow-up reminders. A system that only forwards user questions to a generic cloud chatbot is therefore not sufficient for a safe and explainable healthcare prototype.

This thesis presents the updated implementation of a Smart Medical QA & Consultation System that combines Retrieval-Augmented Generation (RAG), GraphRAG-oriented retrieval, Neo4j knowledge graph querying, and Agentic AI. The current source code is organized as a modular web application rather than a single chatbot script. The backend is implemented with FastAPI and exposes separate routes for health checking, Ollama proxying, graph retrieval, agent consultation, streaming responses, and medical-record analysis. The frontend is a browser-based interface containing pages for dashboard, chat, document upload, health news, and medication reminders.

The first core pillar of the system is a retrieval-grounded medical knowledge layer. Repository abstractions allow the agent and API endpoints to query a configured backend without being tightly coupled to a specific storage implementation. Neo4j and GraphRAG helper modules provide graph context retrieval, source metadata, evidence formatting, and answer synthesis. In parallel, the system integrates verified, custom-built knowledge graph artifacts containing a comprehensive medical ontology of clinical documents, text chunks, normalized entities, semantic mentions, and relations. These artifacts are produced by custom knowledge-graph modules for document chunking, entity extraction, relation extraction, predicate normalization, ontology utilities, and Neo4j ingestion. This design helps the system answer from retrieved context instead of relying only on the parametric memory of the language model.

The second pillar is a robust ReAct-style medical agent. The agent layer contains intent routing, retrieval planning, tool execution, ReAct parsing, parse-retry prompts, loop protection, forced finalization, confidence estimation, and streaming event generation. The available tools include graph retrieval and pill-image lookup, so the model can act on external evidence before writing the final response. The router also detects social turns and simple deterministic paths, allowing the system to avoid expensive agent execution when a full reasoning loop is unnecessary.

The third pillar is multi-format clinical-record processing. The `medical_records` package supports PDF and Excel extraction, laboratory-value parsing, reference-range loading, on-form comparison, abnormal-result classification, medication-suggestion extraction, pill-image storage, and LLM-based advisory summaries. This turns the project from a pure chat application into a more complete clinical-support prototype that can analyze user-provided laboratory reports.

The fourth pillar is operational readiness and usability. Centralized settings manage Ollama, OpenRouter-compatible configuration, Neo4j, vector-store parameters, rate limiting, CORS, and agent behavior. The backend includes cache and Neo4j connection-pooling utilities, health/readiness endpoints, and tests for routes, services, repositories, settings, cache behavior, intent routing, ReAct logic, and evaluation utilities. The web UI supports local reminder scheduling through browser storage, making it possible for users to connect consultation output with medication-follow-up planning.

Overall, the updated system demonstrates a private, extensible, and explainable architecture for localized medical consultation. It does not claim to replace clinicians or provide certified diagnosis. Instead, it shows how local LLMs, graph retrieval, deterministic validation, document parsing, and user-facing workflows can be combined to reduce hallucination risk and improve traceability in a research-oriented CDSS prototype.

Keywords: Clinical Decision Support System (CDSS), Retrieval-Augmented Generation, GraphRAG, ReAct Agent, Neo4j, Ollama, Medical Record Analysis, Explainable AI.

Vương Lê Bình Dương
(*Signature and full name*)

TABLE OF CONTENTS

CHAPTER 1. INTRODUCTION.....	1
1.1 Motivation	1
1.2 Objectives and Scope of the Graduation Thesis	2
1.3 Tentative Solution	2
1.4 Thesis Organization.....	3
CHAPTER 2. REQUIREMENT SURVEY AND ANALYSIS	4
2.1 Status survey	4
2.2 Functional Overview.....	5
2.2.1 Detailed use case diagram	5
2.2.2 System sequence diagrams	8
2.3 Functional description.....	11
2.3.1 Description of UC-01: Consult Medical Agent	11
2.3.2 Description of UC-02: Upload & Analyze Medical Record	11
2.3.3 Description of UC-03: Manage Medication Reminders	12
2.3.4 Description of UC-04: Clinical Calculators and Interaction Safety Check	12
2.3.5 Description of UC-05: View Medical News	13
2.3.6 Description of UC-06: Manage Knowledge Graph.....	13
2.4 Non-functional requirement	13

CHAPTER 3. THEORETICAL BACKGROUND AND TECHNOLOGIES.....	15
3.1 Knowledge Graphs and Labeled Property Graphs (LPG).....	15
3.1.1 RDF vs. Labeled Property Graph (LPG).....	15
3.1.2 Neo4j as the Graph Database Engine	16
3.2 Retrieval-Augmented Generation (RAG) and GraphRAG	19
3.2.1 Classical Vector RAG	19
3.2.2 The Cognitive Limitations of Vector RAG	20
3.2.3 The Paradigm Shift to GraphRAG.....	20
3.3 Agentic AI and Dialog State Orchestration	25
3.3.1 Hybrid Intent Routing Architecture	25
3.3.2 The ReAct (Reasoning and Acting) Framework	26
3.3.3 Agent Safeguards: Loop-Guards and Format Recovery	27
3.3.4 Stateful Dialog Orchestration and Simplified Routing	28
3.4 Advanced Prompt Engineering Strategies.....	29
3.4.1 Role-based Prompting and Persona Assignment.....	29
3.4.2 Negative Prompting and Constrained Generation	29
3.4.3 Chain-of-Thought (CoT) integration via ReAct	29
3.4.4 Few-Shot Formatting and Error Steering	29
3.5 Technical Infrastructure and Development Stack Justification	30
3.5.1 FastAPI Backend (Python 3.10+)	30
3.5.2 Local LLM Hosting	30
3.5.3 High-Speed Unstructured Data Extractors.....	30
3.5.4 Localized Clinical Calculators and Physiological Evaluation	30

CHAPTER 4. DESIGN, IMPLEMENTATION, AND EVALUATION.....	32
4.1 Architecture design.....	32
4.1.1 Software architecture selection	32
4.1.2 Overall design.....	32
4.1.3 Detailed package design.....	33
4.2 Detailed design.....	35
4.2.1 User interface design	35
4.2.2 Layer design	35
4.2.3 Database design.....	36
4.2.4 API Endpoints and JSON Contract Specifications	36
4.2.5 Core Procedural Algorithms	38
4.3 Application Building	40
4.3.1 Libraries and Tools	40
4.3.2 Achievement.....	41
4.3.3 Illustration of main functions	41
4.3.4 Performance Latency Optimizations	45
4.4 Testing and Evaluation	46
4.4.1 Language Model Implementations and Backend Configurations.....	46
4.4.2 Software Regression Tests	46
4.4.3 Evaluation Datasets.....	46
4.4.4 New Comparative Evaluation Method.....	47
4.4.5 Observed Evaluation Results	47
4.4.6 Quantitative System Comparison: GraphRAG vs. Direct LLM.....	48

4.4.7 Evaluation Limitations.....	48
4.4.8 Complementary Evaluation Scripts	48
4.5 Deployment	49
CHAPTER 5. SOLUTION AND CONTRIBUTION.....	51
5.1 Modular Medical CDSS Architecture.....	51
5.2 Graph-Grounded Retrieval and Repository Abstraction.....	51
5.3 Robust ReAct Agent with Recovery Controls	52
5.4 Hybrid Intent Routing	52
5.5 Multi-Format Clinical Record Analysis	52
5.6 Medication Tools and Pill-Image Support	53
5.7 Browser-Based Reminder Workflow	53
5.8 Clinical Calculation and Drug-Drug Interaction Tools	53
5.9 Operational Readiness, Configuration, and Testing	53
5.10 Comparative Evaluation Suite for GraphRAG and Direct LLM.....	53
5.11 Summary of Contributions	54
CHAPTER 6. CONCLUSION AND FUTURE WORK.....	55
6.1 Conclusion	55
6.2 Future Work.....	56
REFERENCE	59
APPENDIX	61
A. USE CASE DESCRIPTIONS.....	61
A.1 Use Case UC-01: Consult Medical Agent.....	61
A.2 Use Case UC-02: Import and Analyze Multi-format Lab Report	61

A.3 Use Case UC-03: Manage Medication Schedule and Reminder..... 61

A.4 Use Case UC-04: Clinical Calculators and Interaction Safety Check..... 62

B. CORE SYSTEM PROMPTS..... 63

C. DEPLOYMENT GUIDE 65

D. DATABASE DICTIONARY 68

E. CORE CODE SNIPPETS 69

LIST OF FIGURES

Figure 2.1	General Use Case diagram for the Smart Medical Consultation System .	5
Figure 2.2	Detailed Use Case diagram for UC-01: Consult Medical Agent	6
Figure 2.3	Detailed Use Case diagram for UC-02: Upload & Analyze Medical Record	6
Figure 2.4	Detailed Use Case diagram for UC-03: Manage Medication Reminders .	7
Figure 2.5	Detailed Use Case diagram for UC-04: Clinical Calculators and Interac- tion Safety Check	7
Figure 2.6	Detailed Use Case diagram for UC-05: View Medical News	7
Figure 2.7	Detailed Use Case diagram for UC-06: Manage Knowledge Graph . . .	8
Figure 2.8	Sequence diagram of the complete medical record analysis and advice generation workflow	9
Figure 2.9	Sequence diagram of the ReAct agent consultation and streaming workflow	9
Figure 2.10	Sequence diagram of the browser-based medication reminder and notifi- cation scheduling workflow	10
Figure 2.11	Sequence diagram of the drug-drug interaction safety check workflow .	10
Figure 3.1	LPG Database Schema Meta-Graph showing Node labels, Relationship types, and property schemas	17
Figure 3.2	Interactive Labeled Property Graph (LPG) subgraph visualization repre- senting the clinical relationship path topology for COVID-19	18
Figure 3.3	Detailed LPG Entity Nodes inside the Neo4j database storing attributes: canonical_name, type, and aliases	18
Figure 3.4	LPG Relationship edge (REL) storing properties: predicate, confidence, and source evidence_chunk_id	19
Figure 3.5	Conceptual flow comparison between conventional Vector RAG and GraphRAG-based clinical retrieval	19
Figure 3.6	Detailed Flowchart of the Local Graph-First Retrieval Pipeline	24
Figure 3.7	Layered architecture of the Hybrid Intent Routing gateway	26
Figure 3.8	ReAct reasoning loop workflow with parser healing and loop-guard safeguards	28

Figure 4.1 Runtime CDSS pipeline from user input to streamed, evidence-grounded response	33
Figure 4.2 System overall package dependency architecture	33
Figure 4.3 UML Class Diagram of core business services and agent components . .	34
Figure 4.4 Sequence diagram illustrating the safeguarded ReAct reasoning loop with parsing format healer and loop-guard checks	35
Figure 4.5 Interactive landing page of the localized Smart Medical QA & Consultation System	42
Figure 4.6 Real-time streamed chat interface showing the ReAct agent consultation and reasoning panel	42
Figure 4.7 Medical record upload interface showing Excel tabular benchmarking and PDF text analysis results	43
Figure 4.8 Weekly medication intake scheduling calendar dashboard with background alarms	44
Figure 4.9 Deterministic clinical calculators interface for BMI and kidney function benchmarking	44
Figure 4.10 Health Dashboard and Medical News feed interface showing localized clinical advisories	45

LIST OF TABLES

Table 2.1	Comparative analysis of medical retrieval paradigms	4
Table 4.1	Neo4j Node Labels and Properties	36
Table 4.2	Neo4j Relationship Types	36
Table 4.3	Libraries and tools used in the system	40
Table 4.4	Latency and stability optimization strategies used in the implemented system	45
Table 4.5	Software regression test summary	46
Table 4.6	Primary metrics in the comparative evaluation framework	47
Table 4.7	Observed aggregate evaluation results from generated reports in <code>eval/</code> .	47
Table 4.8	Quantitative comparison results between GraphRAG (Custom KG) and Direct LLM (Baseline) on 50 clinical QA cases	48
Table 5.1	Summary of system contributions, gaps addressed, and future directions .	54
Table D.1	Node schema in Neo4j Knowledge Graph	68
Table D.2	Relationship schema in Neo4j Knowledge Graph	68

LIST OF ABBREVIATIONS

Abbreviation	Full Expression
CDSS	Clinical Decision Support System
RAG	Retrieval-Augmented Generation
GraphRAG	Graph Retrieval-Augmented Generation
LLM	Large Language Model
SLM	Small Language Model
KG	Knowledge Graph
RRF	Reciprocal Rank Fusion
XAI	Explainable Artificial Intelligence
NER	Named Entity Recognition
API	Application Programming Interface

CHAPTER 1

INTRODUCTION

In recent years, the rapid advancement of Artificial Intelligence (AI) and Large Language Models (LLMs) has opened up revolutionary paradigms across numerous domains, with the clinical and healthcare sectors being among the most promising yet highly sensitive areas of application. However, deploying general-purpose LLMs in medical consultation faces critical bottlenecks. Medical information requires absolute precision; a minor inaccuracy or factual hallucination can lead to life-threatening clinical consequences. Standard generative models frequently suffer from "hallucinations", lack explainable tracing to credible medical literature, and cannot keep pace with rapidly updating pharmacological databases.

Furthermore, local data privacy regulations and patient confidentiality laws present a severe constraint on utilizing commercial, cloud-based LLM APIs. Consequently, there is an urgent research and engineering demand to build localized, private, and structurally-grounded intelligent consultation systems that can analyze clinical data, cross-reference medical literature, and interact with patients in a reliable, secure, and explainable manner.

This thesis presents the design, implementation, and evaluation of a Smart Medical QA & Consultation System utilizing Graph Retrieval-Augmented Generation (GraphRAG) and Agentic AI.

1.1 Motivation

The primary challenge of utilizing LLMs in healthcare lies in ensuring factuality and grounding. Conventional text retrieval methods (Vector RAG) index clinical documents into high-dimensional vector spaces and match query embeddings to retrieve relevant chunks [1]. While effective for finding superficial semantic matches, Vector RAG fails to capture the underlying structural relationships between clinical concepts, such as drug-to-drug interactions, symptom-to-disease causality, and anatomic dependencies.

To overcome these limitations, Graph Retrieval-Augmented Generation (GraphRAG) has emerged as a state-of-the-art framework [2]. By structuring clinical literature into a relational Knowledge Graph (KG) where nodes represent medical entities (e.g., diseases, drugs, symptoms, lab tests) and edges represent normalized clinical predicates (e.g., TREATS, CAUSES, CONTRAINDICATED_FOR), the system preserves structural and relational contexts [3]. Integrating a high-density, purified medical knowledge graph allows for highly precise context retrieval.

Moreover, medical consultation is not a simple one-step search. It requires dynamic, step-by-step reasoning. An intelligent agent utilizing the Reasoning and Acting (ReAct) paradigm can iteratively formulate thoughts, call specific tools (such as searching the custom Neo4j

graph, querying clinical document extractors, or checking pill databases), observe findings, and recursively refine its diagnostic suggestions [4]. Deploying such an agentic loop locally ensures complete patient data confidentiality while maintaining an explainable chain of thought. This combination of structural GraphRAG and Agentic AI serves as the core motivation for this graduation thesis.

1.2 Objectives and Scope of the Graduation Thesis

The primary objective of this graduation thesis is to design, implement, and evaluate a local-first intelligent medical consultation prototype. The system is designed to process clinical records, perform structured medical question-answering, provide medication visual checks, and manage prescription reminders while minimizing dependence on external cloud APIs in the default deployment.

To achieve this objective, the scope of this graduation thesis is defined across the following core boundaries. Medical Knowledge Graph Construction focuses on building and deploying a localized clinical knowledge graph on Neo4j, with scripts for ingestion, entity extraction, relation extraction, predicate normalization, and validation against evaluation datasets. The Graph-First Hybrid Retrieval Engine implements a retrieval pipeline that prioritizes local Neo4j/custom KG evidence through Cypher path traversals, neighborhood expansion, and repository abstractions, while keeping GraphRAG helper modules available as adapters when needed. The Resilient ReAct Agent Design focuses on implementing a robust reasoning-acting agent loop powered by a local LLM backend, reinforced with custom loop-guards to prevent infinite tool invocations and format-recovery parsers to correct syntactical LLM errors. The Multi-Format Clinical Document Analyzer comprises a pipeline that extracts text from medical files (PDFs and Excel sheets) via PyMuPDF and openpyxl, extracts lab results, automatically benchmarks them against printed reference ranges, and generates customized, graph-grounded consultations. The Visual Medication Check and Scheduling focuses on enriching suggested medications with pill images from a local database and enabling automated scheduling for prescription intake reminders. Finally, the Responsive User Interface comprises developing a responsive, clean, and intuitive Web UI to allow patients and clinicians to chat with the agent, upload medical files, view structural citations, and manage schedules.

1.3 Tentative Solution

To fulfill the objectives and scope, this thesis proposes an offline-first, highly-integrated system architecture. The proposed framework is structured into four primary layers: first, the Presentation Layer is built using lightweight, high-performance Vanilla Web technologies (HTML5, responsive CSS3, and modern asynchronous Javascript) to provide live streamed chat responses, interactive document upload logs, and pill identification cards; second, the Business Logic Layer is developed in Python using the FastAPI framework to handle high-concurrency asynchronous endpoints, where the core orchestration consists of the `AgentService` executing the custom `ReActAgent` loop, and the `MedicalRecordService` managing document ex-

traction and automatic lab indicator benchmarking; third, the Data Access Layer manages the retrieval pipelines, resolving structured clinical queries by querying the Neo4j graph database and repository abstractions, while prototype user and session data is handled through local JSON-based storage; and fourth, the Infrastructure and LLM Engine Layer runs entirely offline, combining a local Neo4j Docker container and a local runtime hosting open-source models (such as Llama-3.2-3B) to execute reasoning steps and synthesize final clinical answers. This tentative architecture guarantees data confidentiality, enforces structural factual grounding via Neo4j KG matching, and optimizes runtime speed by purging generic nodes from the retrieval loop.

1.4 Thesis Organization

The remainder of this graduation thesis is organized systematically. Chapter 2 (Requirement Survey and Analysis) conducts a status survey comparing standard search, Vector RAG, and GraphRAG, defining general and detailed use cases, analyzing system sequence diagrams, and listing functional and non-functional requirements. Chapter 3 (Theoretical Background and Technologies) presents the fundamental concepts of Knowledge Graphs, Neo4j, Vector RAG, GraphRAG, the ReAct reasoning paradigm, and dialog state orchestration, while justifying the selection of local open-source LLMs and FastAPI. Chapter 4 (Design, Implementation, and Evaluation) details the clean architecture layers, package layouts, class structures, sequence diagrams, database schemas, and presents system testing cases, integration benchmarks, and local service deployment. Chapter 5 (Solution and Contribution) highlights the core scientific and engineering contributions of this work, focusing on the medical graph retrieval layer, ReAct loop-guards, format-recovery parsers, the multi-format clinical table parser, and the updated comparative evaluation methodology. Finally, Chapter 6 (Conclusion and Future Work) summarizes the achievements of the thesis, discusses technical limitations, and outlines future research paths, including hybrid rerankers and asynchronous task queues.

CHAPTER 2

REQUIREMENT SURVEY AND ANALYSIS

This chapter conducts a detailed requirement survey and system analysis for the proposed smart medical QA and consultation platform. It establishes the current state of retrieval technologies in the medical field, maps out the general and detailed use cases, details core use case specifications, and outlines the functional and non-functional requirements governing the system's runtime environment.

2.1 Status survey

Providing reliable medical information requires a highly specialized approach to information retrieval. Traditional and contemporary systems in the medical QA domain fall into three major paradigms, each having distinct strengths and critical limitations:

Paradigm	Strengths	Weaknesses
Traditional Search (e.g., Google Search)	- Extremely fast access to massive indexed web resources. - High availability.	- High information noise and commercial bias. - Generic results lacking personalized clinical context. - Risk of misinterpretation by laypersons.
Standard Vector RAG	- Semantic matching of patient queries to textbook chunks. - Minimizes basic textual hallucinations.	- Lacks structured clinical connections. - Fails to cross-reference multi-hop dependencies (e.g., drug interactions, symptom chains). - High risk of retrieving irrelevant context due to semantic overlaps.
GraphRAG (Neo4j + Custom KG)	- Preserves precise clinical relationships. - Maps multi-hop pathways (Disease → Symptom → Drug). - Pure offline, private processing capability.	- Higher upfront complexity in constructing and indexing the knowledge graph. - Requires active pruning to prevent generic entity clutter (e.g., "Other" type nodes).

Table 2.1: Comparative analysis of medical retrieval paradigms

Table 2.1 shows that graph-oriented retrieval is the most suitable paradigm for the thesis goal because it preserves explicit clinical relationships while supporting local-first processing. Based on this comparative status survey, the proposed system addresses these gaps by using verified custom KG artifacts in Neo4j for direct factual queries, orchestrated by a local ReAct agent to improve privacy, reasoning transparency, and clinical explainability. GraphRAG helper modules are retained as retrieval adapters, but the implemented consultation flow prioritizes local repository-backed evidence.

2.2 Functional Overview

The system's functional architecture is designed to accommodate two primary actors: the User/Patient seeking clinical consultation and the Medical Admin/Expert managing system resources.

The general use case diagram structures the core capabilities of the system. The two primary actors are defined as follows: the User/Patient (Actor) interacts with the platform to consult the agent, upload and analyze personal health files, view drug imagery, and manage prescription calendars; whereas the Medical Admin/Expert (Actor) oversees system operations, reviews parsed medical record histories, and manages backing database configurations.

The main use cases include: 1. **UC-01: Consult Medical Agent**: The primary interactive channel where the user sends prompts, and the ReAct Agent streams real-time reasoning thoughts, triggers GraphRAG tools, and provides final clinical citations. 2. **UC-02: Upload & Analyze Medical Record**: Allows users to upload clinical PDFs or Excel sheets. The system extracts raw text, executes local indicator benchmarking, and builds customized advice. 3. **UC-03: Manage Medication Reminders**: Translates clinical suggestions into actionable intake schedules with automated browser alarms. 4. **UC-04: Clinical Calculators and Interaction Safety Check**: Enables users to compute deterministic metrics (BMI, eGFR) and verify drug-drug interaction safety. 5. **UC-05: View Medical News**: Allows users to read health articles and updates. 6. **UC-06: Manage Knowledge Graph**: Allows administrative users to apply schemas and import clinical knowledge artifacts.

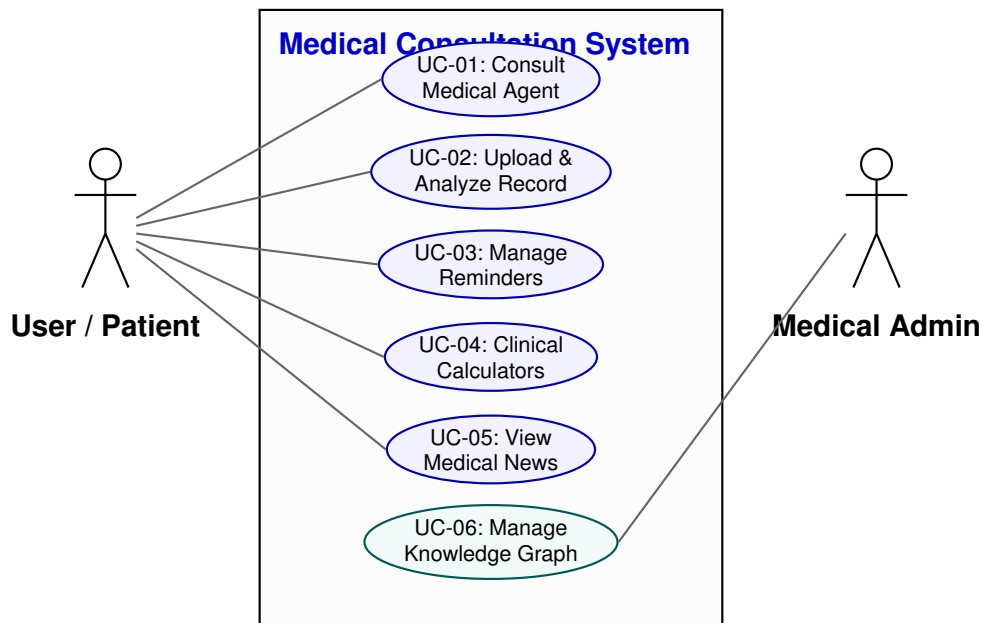


Figure 2.1: General Use Case diagram for the Smart Medical Consultation System

2.2.1 Detailed use case diagram

To represent the sub-systems and detailed relationships of the platform, the primary use cases are decomposed into specific sub-use cases. The following detailed Use Case diagrams

illustrate these breakdowns:

1. UC-01: Consult Medical Agent Detailed Use Case This use case maps the dynamic query-handling flow. As shown in Figure 2.2, the patient interacts with the main chat utility, which includes Cypher path retrieval, neighborhood expansion, and tool execution.

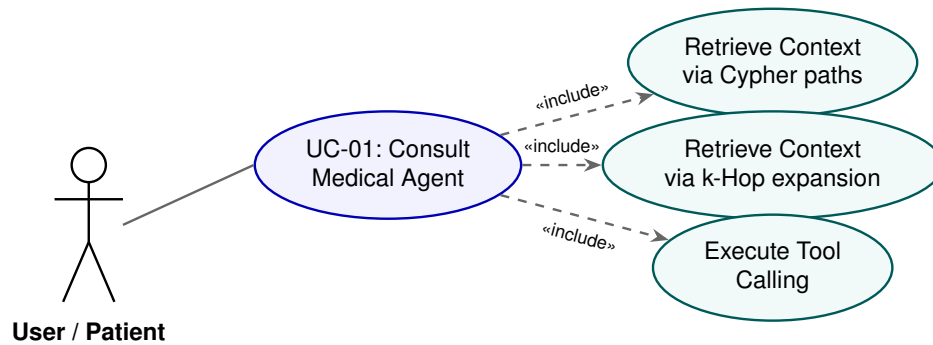


Figure 2.2: Detailed Use Case diagram for UC-01: Consult Medical Agent

2. UC-02: Upload & Analyze Medical Record Detailed Use Case This use case maps the medical report analysis subsystem. As shown in Figure 2.3, the patient uploads a document while the system extracts text, benchmarks numeric indicator values against stored limits, and generates RAG-grounded clinician advice. The medical admin can review these processed records.

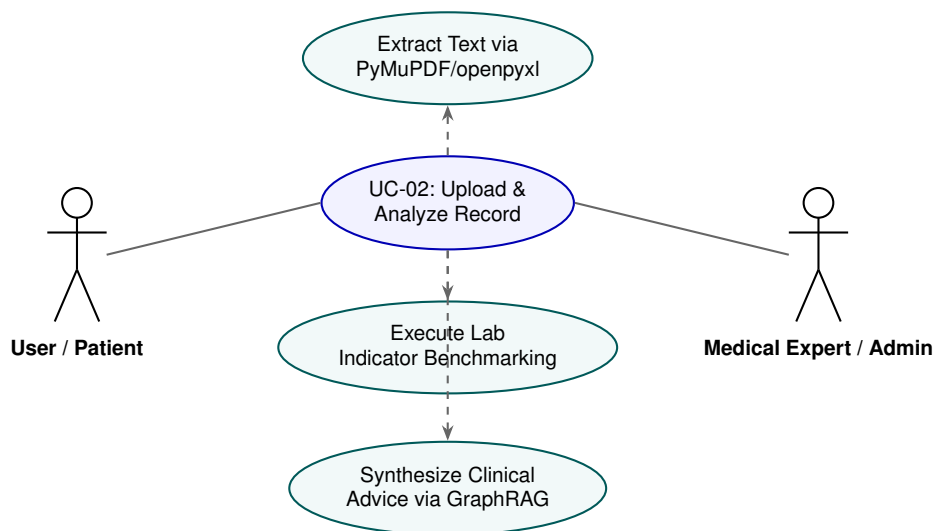


Figure 2.3: Detailed Use Case diagram for UC-02: Upload & Analyze Medical Record

3. UC-03: Manage Medication Reminders Detailed Use Case This use case maps the browser-based intake planning workflow. As shown in Figure 2.4, patients translate advice cards into calendar reminder cards, schedule intake times, log daily adherence, and view their weekly intake grids.

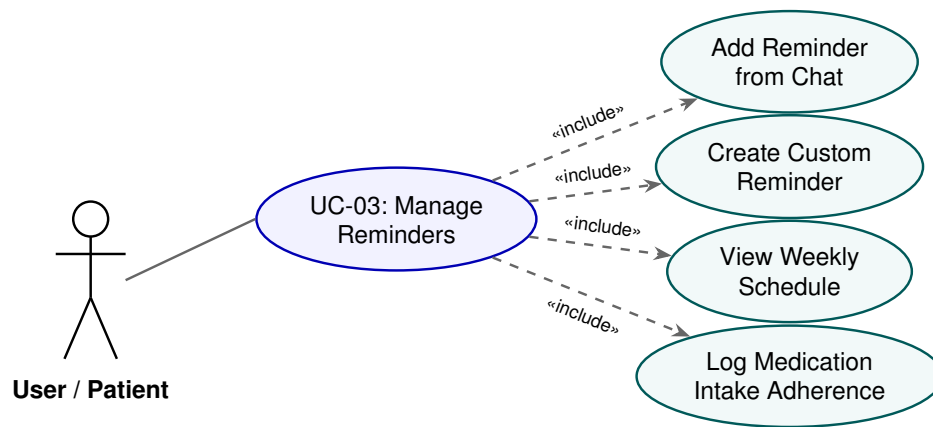


Figure 2.4: Detailed Use Case diagram for UC-03: Manage Medication Reminders

4. UC-04: Clinical Calculators and Interaction Safety Check Detailed Use Case This use case maps the deterministic clinical calculations and drug interaction check flow. As shown in Figure 2.5, patients can calculate physiological metrics like BMI, estimate kidney function (eGFR), or check potential drug-to-drug interactions query-matched from the Neo4j database.

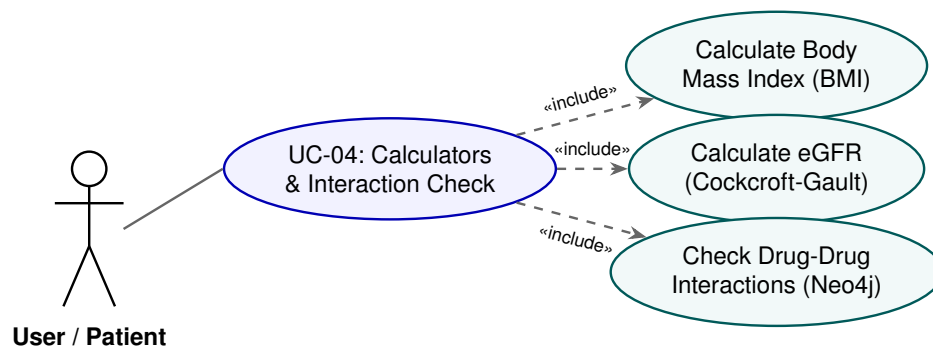


Figure 2.5: Detailed Use Case diagram for UC-04: Clinical Calculators and Interaction Safety Check

5. UC-05: View Medical News Detailed Use Case This use case covers browsing health guidelines, reading localized health articles, and viewing medical advisories synchronized from the Knowledge Graph. As shown in Figure 2.6, patients interact with the news dashboard to access these verified resources.

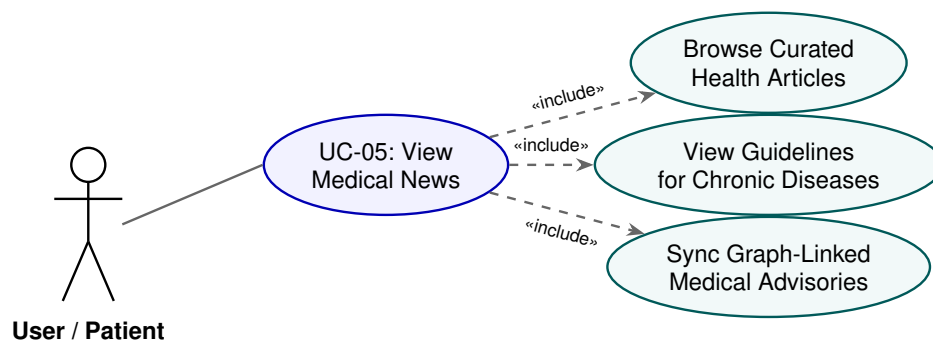


Figure 2.6: Detailed Use Case diagram for UC-05: View Medical News

6. UC-06: Manage Knowledge Graph Detailed Use Case This use case governs the administrative workflows for configuring the persistent clinical database. As shown in Figure 2.7, the Medical Admin utilizes administrative endpoints to ingest documents, apply ontology schemas, prune nodes and relations, and monitor the overall database ingestion status.

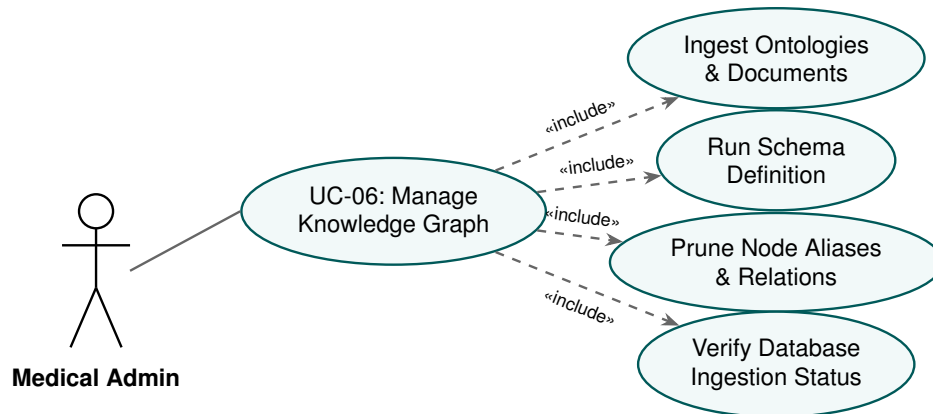


Figure 2.7: Detailed Use Case diagram for UC-06: Manage Knowledge Graph

2.2.2 System sequence diagrams

To detail the step-by-step transaction execution across the software modules, Figure 2.8 illustrates the interaction sequence between the user, the web-based presentation layer, the FastAPI server controllers, the medical record extraction service, the agent reasoning logic, and the Neo4j Custom KG. The system sequence diagrams model the detailed runtime message exchange and transaction flows across actors and architectural components for the core workflows.

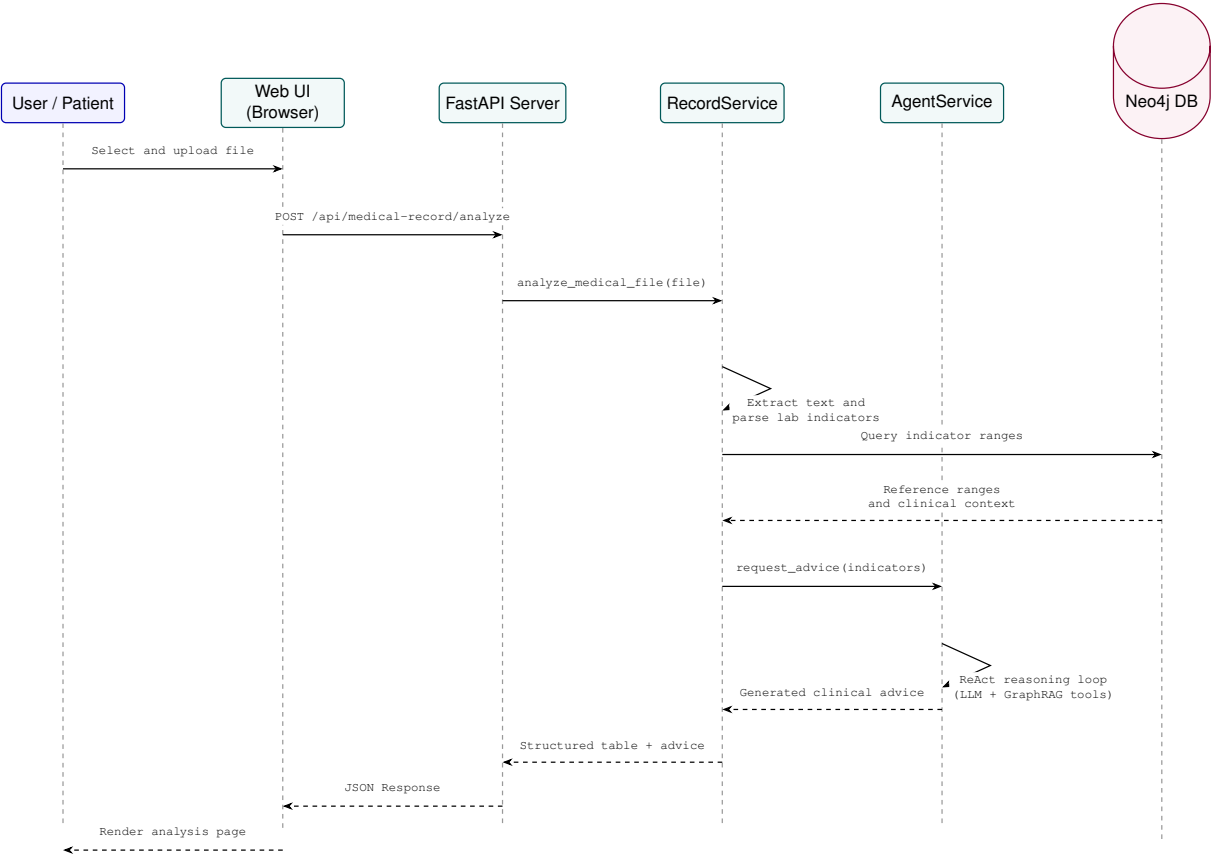


Figure 2.8: Sequence diagram of the complete medical record analysis and advice generation workflow

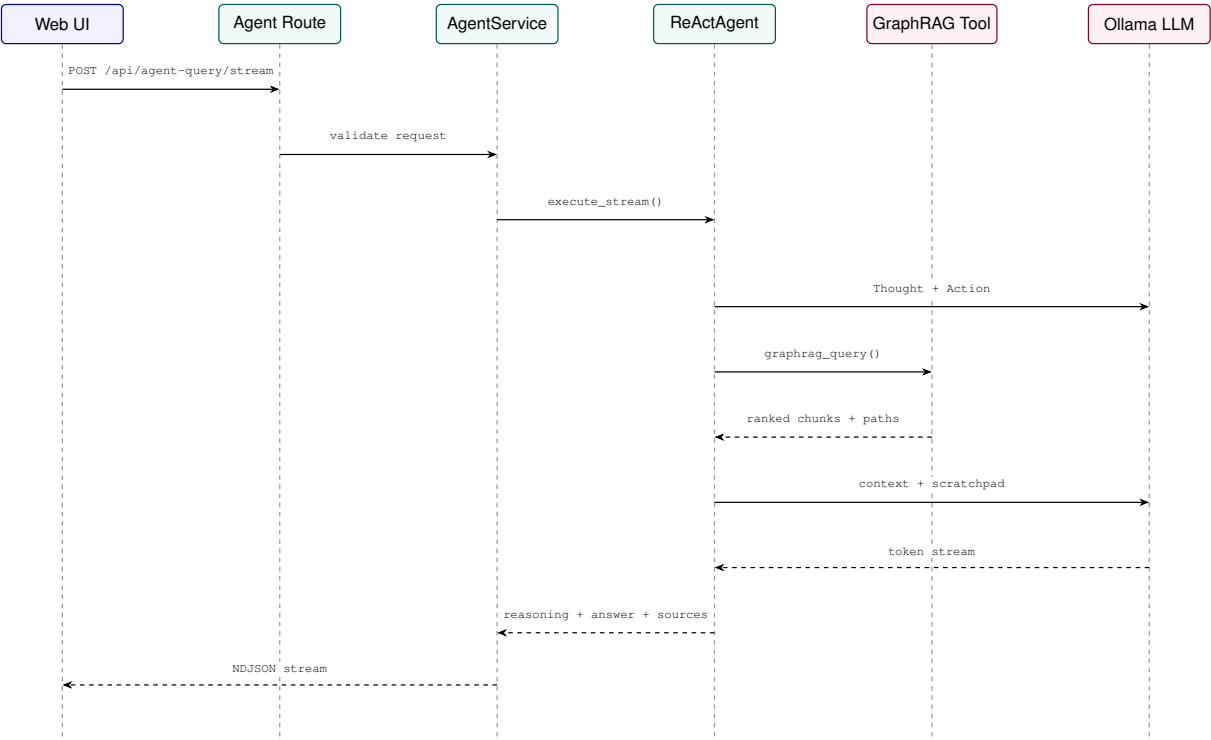


Figure 2.9: Sequence diagram of the ReAct agent consultation and streaming workflow

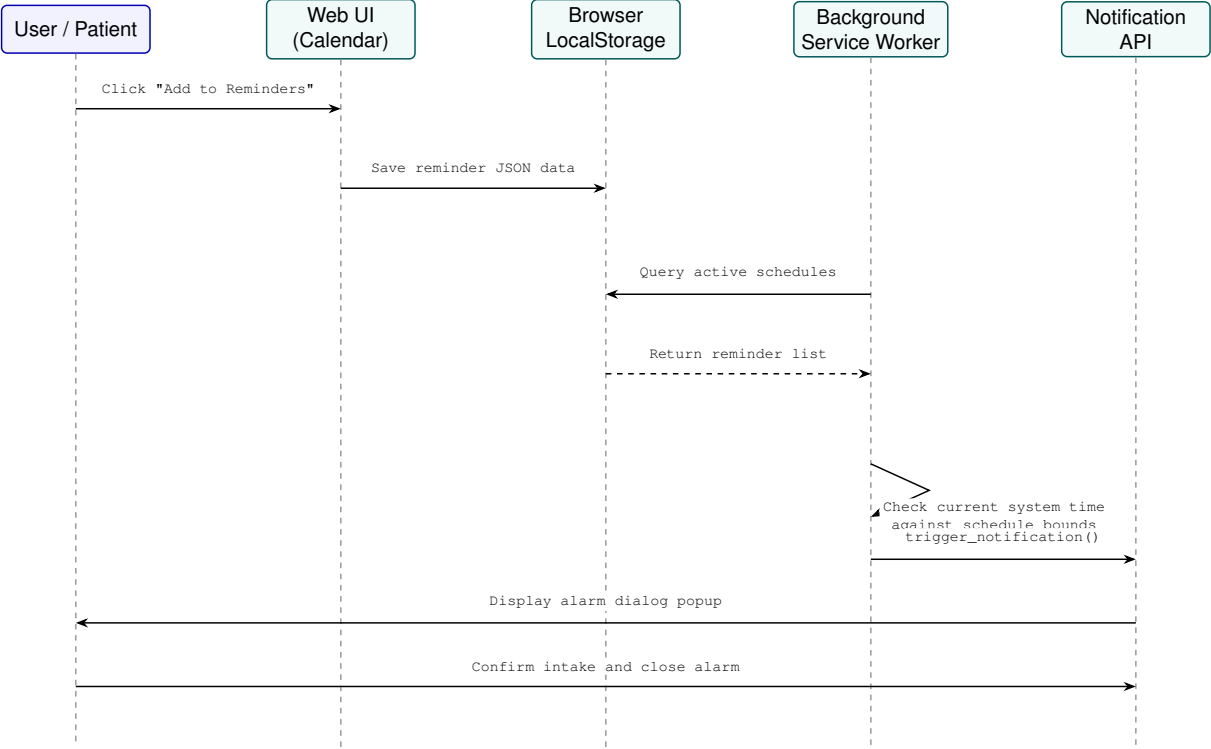


Figure 2.10: Sequence diagram of the browser-based medication reminder and notification scheduling workflow

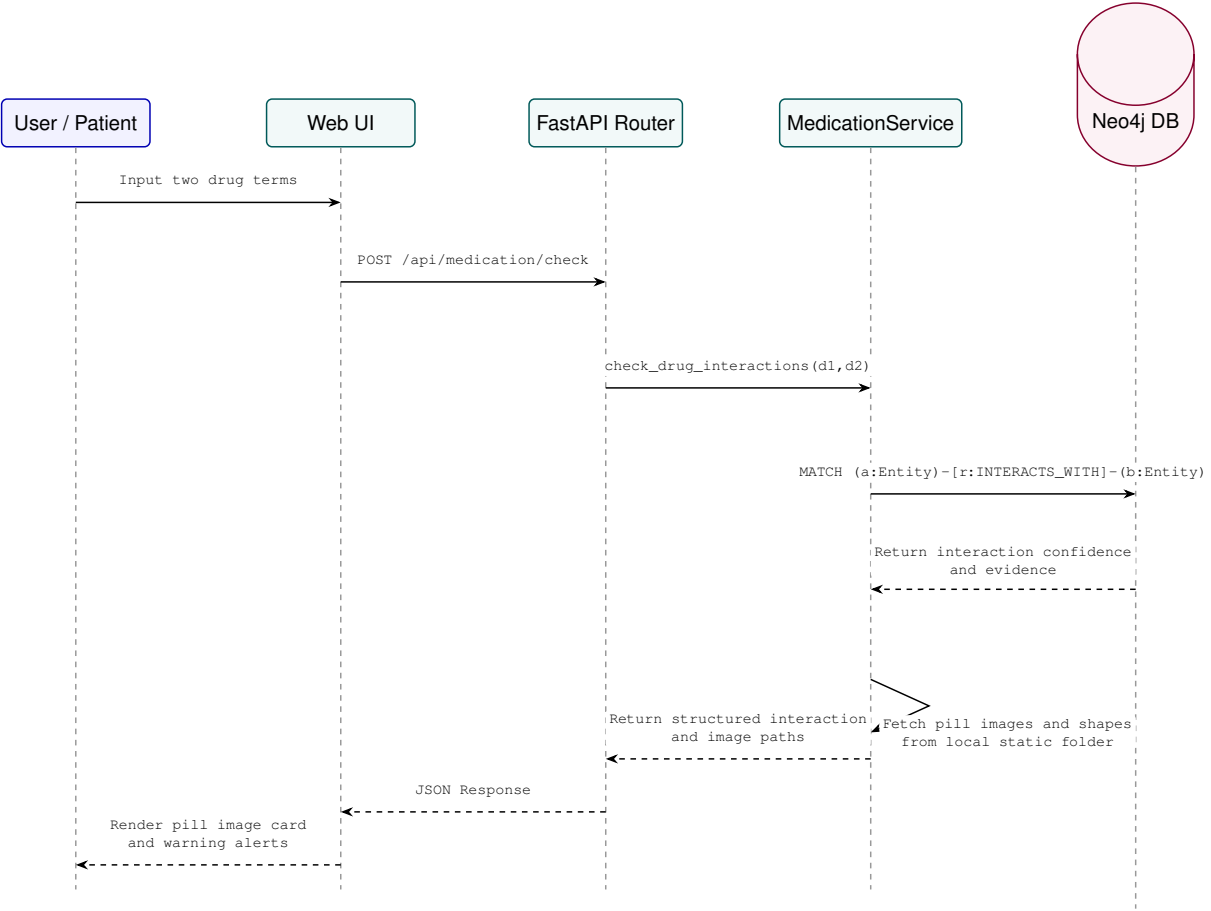


Figure 2.11: Sequence diagram of the drug-drug interaction safety check workflow

2.3 Functional description

The following six use cases represent the critical operational paths of the system and are detailed using standard software engineering specifications (their corresponding execution sequence diagrams are grouped consecutively in Section 2.2.2).

2.3.1 Description of UC-01: Consult Medical Agent

The primary actor is the User / Patient. In this use case, the user inputs a clinical question, and the system executes a ReAct reasoning loop to search the knowledge graph and stream verified clinical answers with source citations. The pre-conditions are that the local Neo4j database is active, and the post-conditions are that the user receives a detailed explanation with relevant entity connections and source citations shown on the UI.

In the basic flow, the user first enters a query (e.g., "Symptoms of Type 2 Diabetes"). Second, the FastAPI backend receives the request and triggers the `AgentService`. Third, the `ReActAgent` parses the query, initiates reasoning, and outputs a `Thought`. Fourth, the agent decides to call the `graphrag_query` tool with the entity "Type 2 Diabetes". Fifth, the tool queries Neo4j directly, finds matches, and returns text chunks as an `Observation`. Sixth, the agent integrates the observation, formulates a final answer, appends source citations, and sends it to the UI.

In the alternate flow for format recovery, if the LLM generates a malformed tool call pattern in iteration 1, the `AgentService` intercepts the error, runs a fallback query matching the original question, synthesizes the results, and returns a verified fallback answer to prevent application crashes.

The sequence of operations during the ReAct consultation stream is detailed in Figure 2.9 (located in Section 2.2.2).

2.3.2 Description of UC-02: Upload & Analyze Medical Record

The primary actor is the User / Patient. The user uploads a clinical report (PDF/XLSX), and the system automatically parses it, benchmarks lab indicators against reference bounds, and provides graph-supported advice. The pre-conditions require that the uploaded file format is supported (`.pdf`, `.xlsx`, `.xslsm`), and the post-conditions dictate that the system renders a comparative benchmark table with color-coded alerts and displays a graph-supported clinical report.

In the basic flow, the user first selects and uploads an Excel sheet containing blood test indicators. Second, the backend extracts cells using `openpyxl`. Third, the system parses numeric readings and printed reference ranges using the function `compare_extracted_report_on_form`. Fourth, an automated matching engine compares the patient's value to the reference bounds. Fifth, the backend triggers `rag_advice_llm` to fetch Neo4j context relating to any detected abnormal indicators. Sixth, the LLM synthesizes a grounded medical

explanation in Vietnamese (or English) and displays it alongside the benchmark table on the dashboard.

The sequence of operations during the medical record ingestion and RAG-supported advice generation is illustrated in Figure 2.8 (located in the previous section).

2.3.3 Description of UC-03: Manage Medication Reminders

The primary actor is the User / Patient. Users customize medication reminder schedules extracted from their clinical recommendations to trigger automatic local notification alerts. The pre-conditions are that the medical analysis narrative contains suggested drugs and dosing parameters, and the post-conditions are that the reminder schedules are stored in browser local storage and rendered in the weekly schedule view.

In the basic flow, the user first views the consultation results showing suggested medications (e.g., "Metformin 500mg, twice a day"). Second, the user clicks "Add to Reminders". Third, the system parses the drug name, dosage, and frequency, transforming them into specific calendar timestamps. Fourth, the intake calendar is saved in browser local storage. Fifth, the browser's active background worker schedules local visual alarms.

The sequence of reminder scheduling and browser background alarms is detailed in Figure 2.10 (located in Section 2.2.2).

2.3.4 Description of UC-04: Clinical Calculators and Interaction Safety Check

The primary actor is the User / Patient. The user utilizes clinical tools to compute essential healthcare metrics (such as BMI and kidney function eGFR) or queries the database to check safety compatibility between medications. The pre-conditions require that the database connection to Neo4j is active and valid user physiological details are provided, and the post-conditions ensure that mathematically validated diagnostic categories or drug conflict details are rendered on the dashboard screen.

In the basic flow, the user first navigates to the clinical tools workspace. Second, they input weight and height values to calculate BMI, or age, weight, creatinine, and gender metrics to calculate eGFR. Third, the frontend sends request parameters to the deterministic backend calculator endpoints. Fourth, the system processes formulas like CKD-EPI or Cockcroft-Gault and returns instant clinical classifications. Alternatively, the user adds drug search terms to inspect drug-drug interactions. The backend executes a direct Neo4j Cypher match to return safety compatibility and warnings.

The sequence of drug-drug interaction checking and Neo4j safety querying is detailed in Figure 2.11 (located in Section 2.2.2).

2.3.5 Description of UC-05: View Medical News

The primary actor is the User / Patient. The user browses health advisory updates, chronic disease guidelines, and localized medical advisories linked directly with clinical entities in the Knowledge Graph. The pre-conditions are that the local server is running and the database connection is active, and the post-conditions are that curated advisory articles and graph-connected recommendations are rendered on the news dashboard.

In the basic flow, the user first navigates to the medical news feed. Second, the frontend sends a query to the backend guideline router. Third, the system fetches guidelines corresponding to seed chronic conditions (e.g. hypertension, diabetes). Fourth, the backend resolves graph entities to include related disease parameters and advisories. Fifth, the system returns a JSON payload containing guidelines and synced news cards. Sixth, the web UI displays interactive guidelines and localized health cards to the user.

2.3.6 Description of UC-06: Manage Knowledge Graph

The primary actor is the Medical Admin. The administrator manages system ontologies, runs graph schema definitions, imports medical documents, and verifies database ingestion statistics. The pre-conditions are that the administrator has server-level command-line access or is accessing the admin tool, and the local Neo4j instance is running, and the post-conditions dictate that the custom KG schema is updated and document nodes are correctly ingested and linked.

In the basic flow, the administrator first loads the clinical document ontology files (e.g. from the `kg/kg_artifacts` directory). Second, they execute the Neo4j schema initialization script to define fulltext indexes and unique constraint properties. Third, the admin runs the document ingestion parser to chunk, extract entities, normalize predicates, and write the triple records into the Neo4j database. Fourth, the system parses references and writes links between nodes. Fifth, the admin triggers the verification suite to check entity counts and relationship coverage. Sixth, the system returns a status report showing successful graph synchronization.

2.4 Non-functional requirement

To guarantee system stability, local execution feasibility, and absolute privacy, the following non-functional requirements are enforced: First, regarding private and offline execution, the system must run entirely on the user's local machine without sending clinical prompts to external cloud APIs, ensuring compliance with data privacy regulations.

Second, to optimize response streaming speed, the chat API `/api/agent-query/stream` must yield the first reasoning token within 1.5 seconds of receiving the user query, and stream chunks continuously to avoid high perceived latency on local hardware.

Third, for security and rate limiting, the system implements FastAPI dependency injec-

tion rate limiters (`check_rate_limit`) to restrict queries to 30 requests per minute per IP, preventing denial-of-service loops on local LLM runtimes.

Fourth, in terms of hardware feasibility, the local LLM engine (such as Llama-3.2-3B) must execute within standard consumer hardware constraints with an active request timeout limit of 120 seconds.

CHAPTER 3

THEORETICAL BACKGROUND AND TECHNOLOGIES

This chapter provides a rigorous theoretical exposition and technical justification of the methodology utilized to build the offline-first, reliability-oriented clinical question-answering (QA) and consultation prototype. Section 3.1 details the mathematical and structural formulation of Labeled Property Graphs (LPGs) and their implementation via the Neo4j engine. Section 3.2 evaluates standard Vector Retrieval-Augmented Generation (RAG), analyzes its fundamental cognitive limitations, and presents the paradigm shift to GraphRAG under a dual-layer retrieval architecture. Section 3.3 establishes the framework for Agentic AI, focusing on the Reasoning and Acting (ReAct) paradigm, custom runtime safety guards, and simplified stateful dialog orchestration. Section 3.4 details the Advanced Prompt Engineering strategies employed to steer the ReAct models. Finally, Section 3.5 offers a comprehensive technical justification of the localized development stack, emphasizing asynchronous stream processing, local LLM hosting, and privacy-preserving document digestion.

3.1 Knowledge Graphs and Labeled Property Graphs (LPG)

A Knowledge Graph (KG) is a structured, semantically rich representation of real-world information, modeling domain concepts as vertices and their interactions as directed, labeled edges [3]. Formally, a knowledge graph G can be defined as shown in Equation (3.1):

$$G = (E, R, T) \quad (3.1)$$

where $E = \{e_1, e_2, \dots, e_n\}$ is a set of unique entities, $R = \{r_1, r_2, \dots, r_m\}$ is a set of semantic relation types, and $T \subseteq E \times R \times E$ is a set of fact triples in the form of (s, p, o) , representing a subject entity s , a predicate relationship p , and an object entity o .

In the clinical domain, raw, unstructured texts often obscure critical diagnostic pathways. Knowledge graphs normalize these texts into a standardized ontology (such as SNOMED-CT or UMLS translations), ensuring that synonymous concepts (e.g., “*Type 2 Diabetes Mellitus*”, “*T2DM*”, and “*Non-insulin-dependent diabetes*”) map to a single, canonical entity.

3.1.1 RDF vs. Labeled Property Graph (LPG)

While the Semantic Web historically relied on the Resource Description Framework (RDF) where every statement is represented as a flat triple (s, p, o) , modern intelligent applications require high-dimensional attributes (properties) on both nodes and edges to record metadata (such as confidence levels, source file references, and clinical guidelines). This requirement is met by the Labeled Property Graph (LPG) model.

Mathematically, an LPG is defined as a 6-tuple in Equation (3.2):

$$G_{LPG} = (V, E, L_V, L_E, P_V, P_E) \quad (3.2)$$

where V is a finite set of vertices (nodes) representing medical entities or informational chunks; $E \subseteq V \times V$ is a set of directed edges representing clinical or structural relationships; $L_V : V \rightarrow \mathcal{P}(\Sigma_V)$ is a node-labeling function mapping each node to a set of labels from the label alphabet Σ_V (e.g., Entity, Chunk, Document); $L_E : E \rightarrow \Sigma_E$ is an edge-labeling function mapping each relationship to a specific type from the relation alphabet Σ_E (e.g., TREATS, CAUSES, MENTIONS); $P_V : V \times K \rightarrow U$ is a node-property evaluation function mapping a node and a key $k \in K$ to a value $u \in U$ (e.g., storing `canonical_name`, `type`, or `embedding_vector`); and $P_E : E \times K \rightarrow U$ is an edge-property evaluation function mapping a relationship and a key k to a value u (e.g., storing `predicate`, `confidence_score`, or `source_document`).

3.1.2 Neo4j as the Graph Database Engine

This system implements the Labeled Property Graph model using Neo4j, a native graph database engine designed for high-performance traversals. Unlike relational databases (RDBMS) which require index-join operations that scale with table size ($O(\log N)$ or $O(N \log N)$), Neo4j utilizes index-free adjacency. At the storage layer, each node record holds direct physical pointers (memory addresses) to its first relationship record, and relationship records form a doubly linked list linking the source and target node records. Thus, traversing an edge consists of following a direct reference offset in memory. This reduces graph traversal complexity to Equation (3.3):

$$T_{traverse} = O(d) \quad (3.3)$$

where d is the local degree (number of connected edges) of the active node, completely independent of the total number of nodes N in the database.

To interact with this graph, the system utilizes Cypher, a declarative, pattern-matching query language. For clinical diagnostics, traversing relationships must be highly precise. Consider a query to find all active ingredients in medications that treat a specific symptom, while ensuring the relationship confidence exceeds a threshold of 0.80. The pattern is matching as follows:

```
MATCH (symptom:Entity {canonical_name: "Hyperglycemia"})
      <-[:REL {predicate: "CAUSED_BY"}]-(disease:Entity)
MATCH (disease)
      <-[:REL {predicate: "TREATS"}]-(drug:Entity)
WHERE r.confidence > 0.80
RETURN DISTINCT drug.canonical_name AS treatment, r.confidence AS score;
```

This direct, structural query guarantees that the retrieved clinical connections are completely verified and grounded in the database, avoiding the hallucination risks associated with standard vector searches.

In this system, the clinical knowledge base is structured using a specific property graph schema in Neo4j. The nodes belong to three primary labels: `Entity` represents medical concepts such as drugs, diseases, symptoms, or tests, with attributes including `entity_id` (a unique string identifier derived as a hash of its canonical name), `canonical_name` (the unified clinical term in lowercase for exact matching), `type` (storing labels such as DRUG, DISEASE, SYMPTOM, or TEST), and `aliases` (a list of alternative Vietnamese/English synonym names); `Chunk` represents the raw source text block containing the concept, storing `chunk_id` (unique identifier hash), `text` (the raw string segment), and its corresponding dense embedding vector; and `Document` represents the source medical literature or clinical guideline file, storing attributes such as `doc_id`, `filename`, and `filepath`. The relationships link these nodes together, with the most important being the `REL` relationship connecting two `Entity` nodes (which contains specific properties such as `predicate` representing the clinical action like TREATS, CAUSES, CONTRAINDICATED_FOR, INTERACTS_WITH, SIDE_EFFECT_OF, or HAS_SYMPTOM, a numerical confidence weight in the range $[0, 1]$, and an `evidence_chunk_id` string ID referencing the backing `Chunk` node) and the `MENTIONS` relationship connecting a `Chunk` to an `Entity` (which stores the `mention_confidence` property reflecting mapping reliability). The graph database is enriched with the APOC library, which provides utility procedures to execute rapid fulltext index queries and multi-hop Cypher path expansions.

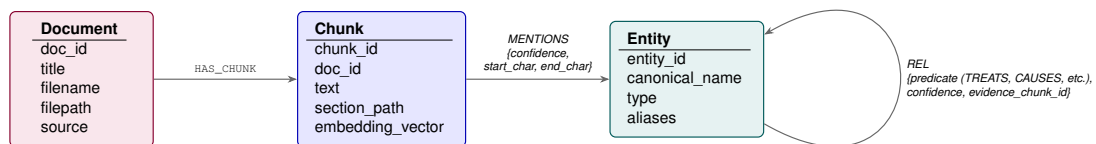


Figure 3.1: LPG Database Schema Meta-Graph showing Node labels, Relationship types, and property schemas

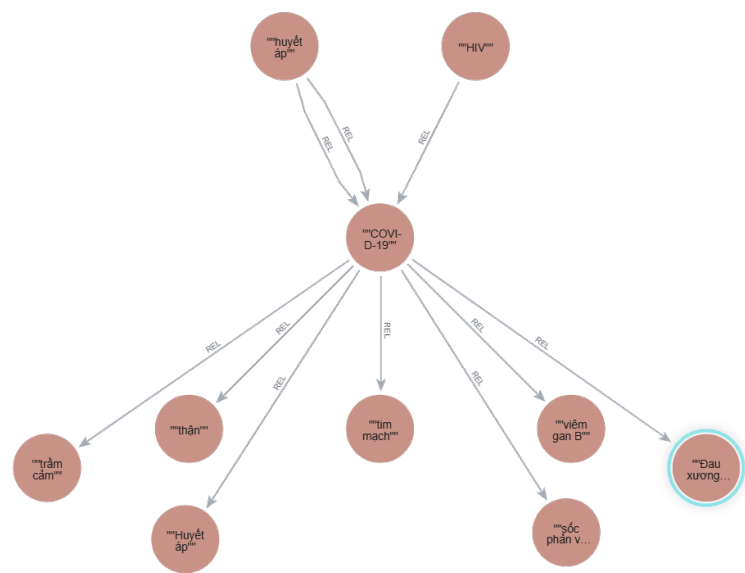


Figure 3.2: Interactive Labeled Property Graph (LPG) subgraph visualization representing the clinical relationship path topology for COVID-19

Node details	
Entity	
Key	Value
<id>	4:6b7d1477-1276-4ebe-8cfe-0462b2bba8b1:28924
aliases	[]
canonical_name	"COVID-19"
created_at	"2026-05-11T03:26:19Z"
entity_id	"ent_99833f7618d90cf6"
type	"Disease"

(a) COVID-19 Entity Node

Node details	
Entity	
Key	Value
<id>	4:6b7d1477-1276-4ebe-8cfe-0462b2bba8b1:29272
aliases	[]
canonical_name	"Pfizer"
created_at	"2026-05-11T03:26:19Z"
entity_id	"ent_e58d5b9fce7a306"
type	"Drug"

(b) Pfizer Drug Entity Node

Node details	
Entity	
Key	Value
<id>	4:6b7d1477-1276-4ebe-8cfe-0462b2bba8b1:29483
aliases	[]
canonical_name	"viêm thần kinh liên sườn"
created_at	"2026-05-11T06:06:47Z"
entity_id	"ent_ed856ed5474710d7"
type	"Disease"

(c) Disease Entity Node

Figure 3.3: Detailed LPG Entity Nodes inside the Neo4j database storing attributes: canonical_name, type, and aliases

Relationship details	
REL	
Key	Value
<id>	5:6b7d1477-1276-4ebe-8cfe-0462b2bba8b1:134796
confidence	0.95
evidence_chunk_id	"chunk_281fb0903e2a583c"
predicate	"TREATS"

Figure 3.4: LPG Relationship edge (REL) storing properties: predicate, confidence, and source evidence_chunk_id

Figure 3.2 visualizes the COVID-19 clinical subgraph used to demonstrate LPG topology. Figure 3.3 details representative Neo4j entity nodes, including the COVID-19 node in Figure 3.3a, the Pfizer drug node in Figure 3.3b, and the disease node in Figure 3.3c. Figure 3.4 then shows how relationship properties preserve predicate, confidence, and evidence provenance for graph traversal.

3.2 Retrieval-Augmented Generation (RAG) and GraphRAG

Figure 3.5 summarizes the conceptual difference between conventional Vector RAG and the proposed GraphRAG-oriented retrieval strategy. The important distinction is that Vector RAG retrieves isolated text fragments based mainly on embedding similarity, whereas GraphRAG first identifies clinical entities, follows explicit graph relationships, and only then composes grounded evidence for the LLM. This design is particularly important in medical consultation because many questions require relational reasoning rather than simple keyword or semantic matching.

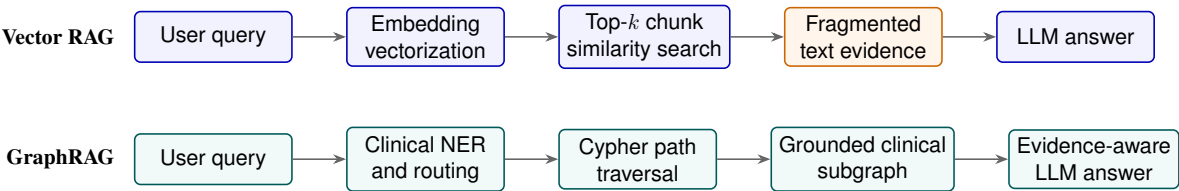


Figure 3.5: Conceptual flow comparison between conventional Vector RAG and GraphRAG-based clinical retrieval

3.2.1 Classical Vector RAG

Traditional Retrieval-Augmented Generation (RAG) processes unstructured text by dividing files into static chunks (e.g., 500-character segments with 100-character overlap), feeding them through a dense embedding model (e.g., a pre-trained transformer), and indexing the resulting

vector representations in a vector database [1]. Given a user query q , the query is embedded into the same vector space, and the system retrieves the top- k nearest chunks. Mathematically, let $\vec{q} \in \mathbb{R}^D$ be the query embedding vector and $\vec{d}_i \in \mathbb{R}^D$ be the document chunk embedding vector. The similarity metric is computed using Cosine Similarity as shown in Equation (3.4):

$$\text{Sim}(\vec{q}, \vec{d}_i) = \cos(\theta) = \frac{\vec{q} \cdot \vec{d}_i}{\|\vec{q}\|_2 \|\vec{d}_i\|_2} = \frac{\sum_{j=1}^D q_j d_{i,j}}{\sqrt{\sum_{j=1}^D q_j^2} \sqrt{\sum_{j=1}^D d_{i,j}^2}} \quad (3.4)$$

3.2.2 The Cognitive Limitations of Vector RAG

Despite its effectiveness in local search, Vector RAG suffers from severe cognitive limitations when applied to clinical reasoning: First, the multi-hop reasoning failure is a structural limitation of vector search to single-step semantic lookups. For example, if a user asks: “*Does drug A interact with the medication prescribed for patient B’s primary kidney condition?*”, a standard vector database matches chunks containing `drug A` or `patient B`, but cannot construct the logical pipeline: `Patient B` $\rightarrow$ `Kidney Condition` $\rightarrow$ `Prescribed Medication` $\rightarrow$ `Interaction with Drug A` due to the lack of relational links needed to transition across distinct nodes.

Second, the loss of structural context occurs because chunking breaks the continuous semantic thread of a document. If a medical table links a specific dosage to a patient’s weight class, a flat text chunker might split the row, separating the causal condition from the clinical recommendation, leading to fragmented or incorrect context.

Third, the lack of global synthesis makes it difficult to address global queries. When a clinician requests a query like “*What are the common side effects of all anti-hypertensive drugs documented in our medical archive?*”, standard Vector RAG fails as it can only fetch specific chunks containing `anti-hypertensive` and `side effects`, leaving the LLM to synthesize fragmented chunks while missing the broader picture.

3.2.3 The Paradigm Shift to GraphRAG

To address these limitations, this system implements a hybrid GraphRAG framework, integrating dense vector matching with structural LPG traversals [2]. The architecture executes queries across two distinct pathways:

a, 1. Local Graph-First Retrieval with Clinical NER and Cypher Path Queries

To ensure precise semantic grounding and eliminate context pollution, the system employs a multi-stage Local Graph-First Retrieval pipeline. Unlike simple keyword matching or unstructured k -hop expansions which retrieve distant, irrelevant nodes, this pipeline targets the specific relational pathways linking the core clinical entities mentioned in the query:

Stage 1: Lightweight LLM-based Clinical NER Extraction When a user query q is submitted, it is first analyzed by a dedicated, lightweight Named Entity Recognition (NER) extractor $f_{NER}(q)$ powered by a quantized local Small Language Model (SLM), such as `qwen2.5:1.5b-instruct` running at temperature 0.0. The SLM is prompted to return a strictly structured JSON array representing the extracted clinical concepts in Equation (3.5):

$$E_{clinical} = \{(name_k, type_k) \mid k = 1 \dots N_E\} \quad (3.5)$$

where each entity name $name_k$ is categorized into one of four clinical entity types $type_k \in \{\text{DRUG, DISEASE, SYMPTOM, TEST}\}$. Extracting clinical entities using a lightweight 1.5B model guarantees near-instant extraction latencies ($\approx 100\text{ms}$) while preventing structural schema hallucination.

Each extracted entity $name_k$ is then mapped to its corresponding node ID $v_k \in V$ in the Neo4j database using a synonym-aware fulltext index search. Neo4j’s Lucene fulltext search matches $name_k$ against both the node’s `canonical_name` and its `aliases` array. If a match is found, it resolves to a unique entity node ID via Equation (3.6):

$$v_k = \text{ResolveNode}(name_k) \quad (3.6)$$

resulting in a set of resolved seed node IDs $V_{seed} = \{v_1, v_2, \dots, v_m\}$.

Stage 2: Cypher Path Query Traversal If the query mentions multiple clinical entities ($|V_{seed}| \geq 2$), the retrieval engine attempts to construct a highly targeted relational path linking them together. Rather than expanding the graph blindly around each node, it executes a targeted Cypher path query to find paths of length 1 to H (maximum hops) that connect any pair of distinct seed entities as defined in Equation (3.7):

$$P_{paths} = \{p \mid p = (u_0 \xrightarrow{r_1} u_1 \xrightarrow{r_2} \dots \xrightarrow{r_h} u_h), u_0, u_h \in V_{seed}, u_0 \neq u_h, h \leq H\} \quad (3.7)$$

The Cypher query syntax utilized for this path discovery is formulated as:

```
MATCH p=(a:Entity)-[:REL*1..2]-(b:Entity)
WHERE a.entity_id IN $ids AND b.entity_id IN $ids AND a.entity_id < b.
      entity_id
WITH p LIMIT 50
UNWIND nodes(p) AS n
UNWIND relationships(p) AS r
WITH collect(distinct n) AS ns, collect(distinct r) AS rs
RETURN ns AS entities, rs AS edges;
```

This operation returns a dense, highly relevant subgraph containing the precise connections between the target concepts (for example, the direct `INTERACTS_WITH` relationship linking two drugs, or the `TREATS` edge connecting a drug to a disease).

If the path query returns a non-empty set of edges, the system extracts the associated text paragraphs via their stored `evidence_chunk_id` properties on the matched edges. These paragraphs represent the exact medical literature proving the connection.

Fallback to k-Hop Expansion If the seed list contains only a single resolved entity ($|V_{seed}| < 2$) or if the Cypher path query returns zero connecting paths due to graph sparsity, the engine dynamically falls back to standard fulltext-seeded k -hop subgraph expansion formulated in Equation (3.8):

$$G_{sub} = \text{ExpandSubgraph}(V_{seed}, H) \quad (3.8)$$

This dual-stage retrieval mechanism ensures that when the patient asks about drug interactions or symptoms of a specific condition, the system injects only the direct, verified medical evidence into the prompt context, avoiding prompt bloat and reducing hallucination.

Soft Subgraph Pruning Layer To eliminate indexing noise, the retrieved raw subgraph undergoes a clinical pruning process:

$$G'_{sub} = \text{PruneSubgraph}(G_{sub}, V_{seed}) \quad (3.9)$$

This operation implements three main rules: first, under regex noise filtering, any entity whose canonical name matches the regular expression `^[0-9\W_]+$` (representing standalone numbers, symbols, or punctuation) is discarded; second, under type and quality filtering, generic concepts with type tags belonging to the set $\{\text{other, generic, noise, document, chunk}\}$ are filtered out; and third, under degree-0 isolated node pruning, any node with zero relationships is removed, unless it is a primary seed node in V_{seed} to prevent isolated, non-relational entities from cluttering the context.

Hybrid Candidate Ranking via RRF Once candidate text chunks are fetched (from paths and direct seed entity mentions), they are ranked across two separate retrieval branches. The first branch is the graph-based rank branch, where chunks are sorted in descending order of their `mention_confidence` property, reflecting structural KG extraction quality. The second branch is the lexical-based rank branch, where chunks are sorted based on character-token overlap. The similarity score is calculated using the overlap coefficient:

$$\text{Overlap}(Q, C) = \frac{|T(Q) \cap T(C)|}{\max(1.0, |T(Q)|^{0.5})} \quad (3.10)$$

where $T(X)$ is the set of unique lowercased alphanumeric tokens of length $2 \leq \text{len} \leq 40$ in string X . The two ranked lists are then fused into a single sorted list using Reciprocal Rank

Fusion (RRF) defined in Equation (3.11):

$$RRF(d) = \sum_{m \in \{graph, lexical\}} \frac{1}{60 + r_m(d)} \quad (3.11)$$

where $r_m(d)$ represents the 1-indexed rank of chunk d in retrieval branch m .

LLM-based Reranking Layer (Optional) To optimize candidate density further, the system can enable an optional neural reranking layer. It sends the top-20 fused chunks alongside the user query to the local LLM with a highly constrained prompt. The LLM is forced to evaluate the chunks and return a JSON array containing only the indices of the most relevant chunks. The final top- k elements are then sliced for prompt injection.

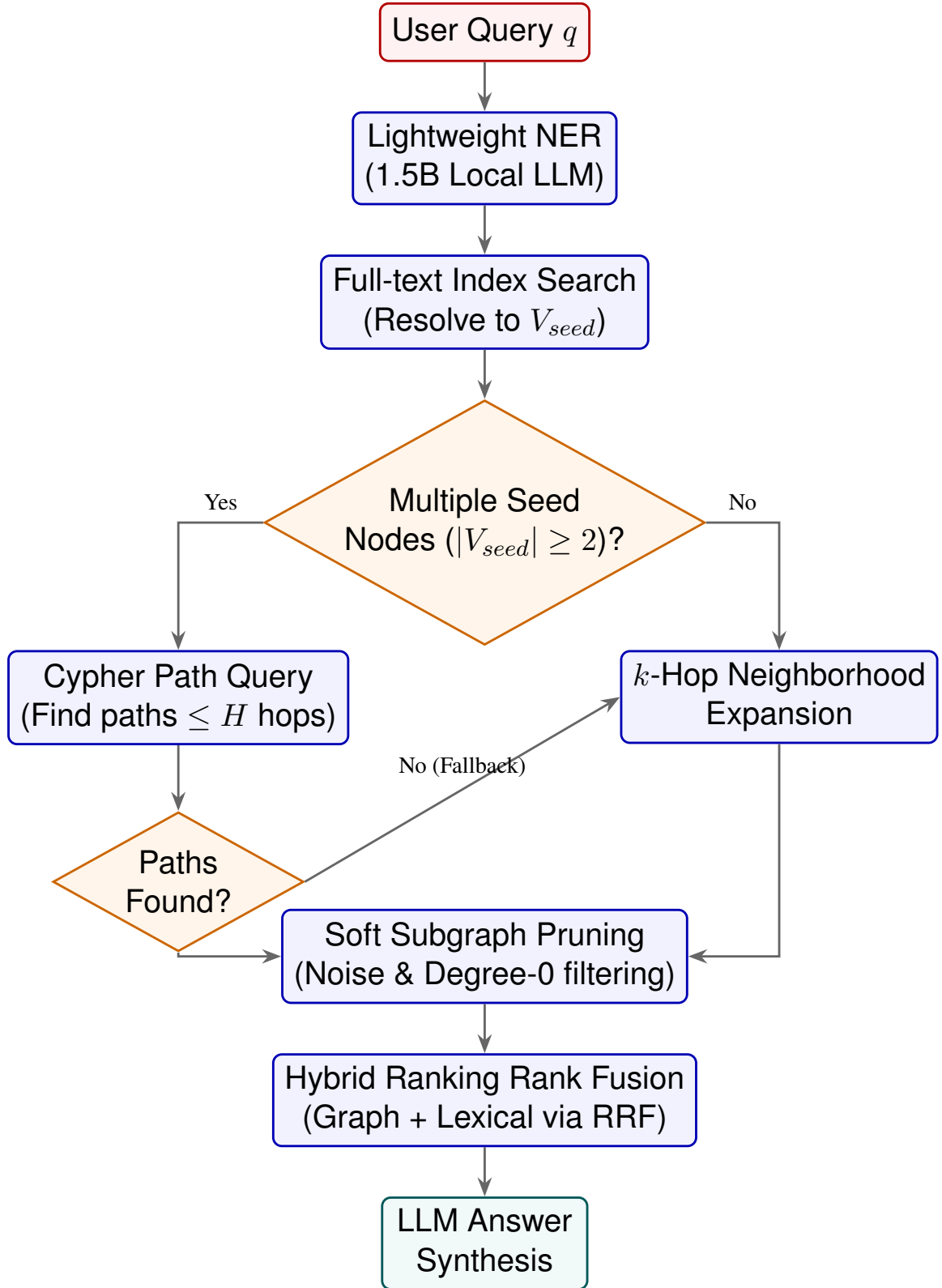


Figure 3.6: Detailed Flowchart of the Local Graph-First Retrieval Pipeline

b, 2. Global GraphRAG (Microsoft GraphRAG Paradigm)

For global, high-level clinical queries, hierarchical community clustering algorithms (such as the Microsoft GraphRAG paradigm) were evaluated. During the offline indexing phase, the entire clinical graph is partitioned into hierarchical communities using the Leiden Algorithm [5],

which optimizes modularity as shown in Equation (3.12):

$$Q_{Leiden} = \frac{1}{2m} \sum_{i,j} \left(A_{ij} - \gamma \frac{k_i k_j}{2m} \right) \delta(\sigma_i, \sigma_j) \quad (3.12)$$

where A_{ij} is the adjacency matrix weight, k_i and k_j are the degrees of nodes i and j , m is the total edge weight, γ is the resolution parameter, and $\delta(\sigma_i, \sigma_j)$ is the Kronecker delta representing community assignment. Next, for each discovered community, a local LLM writes a comprehensive narrative summary detailing the entities, relationships, and key findings within that cluster. Finally, when a global query is executed, the engine runs a MapReduce-style retrieval, identifying the relevant community summaries (Map phase), generating partial answers, and aggregating them into a final clinical synthesis (Reduce phase). While this global community summarization provides high-level synthesis, executing MapReduce queries against local LLMs incurs extreme latencies (often exceeding 30 seconds per request) and requires massive VRAM resources. Consequently, the active query engine bypasses the Microsoft GraphRAG CLI pipeline, routing queries exclusively to the Custom KG to guarantee real-time, low-latency performance under 1.5 seconds.

3.3 Agentic AI and Dialog State Orchestration

Static QA pipelines process inputs in a rigid, linear sequence. However, medical consultations are highly dynamic, requiring agents to ask clarifying questions, correct formatting errors, or dynamically call tools to verify facts. The system uses Agentic AI to manage these multi-turn diagnostic reasoning loops.

3.3.1 Hybrid Intent Routing Architecture

To minimize response latency and avoid invoking heavy language models for simple queries, the system deploys a high-efficiency, layered Hybrid Intent Router as the gateway of the conversation. Given a user query q , the routing gateway $R(q)$ directs the query along two possible execution pipelines based on a hybrid classification mechanism:

If the query does not match obvious social/greetings checks, the system invokes a parallel, specialized Small Language Model (SLM) classification step. The system instantiates a dedicated instance of a 1.5B parameter model (specifically `qwen2.5:1.5b-instruct`) running in parallel on the local server. The SLM is prompted to classify the query's intent into one of two routing tokens as formulated in Equation (3.13):

$$R(q) \in \{\text{graph_first}, \text{global_summary}\} \quad (3.13)$$

where `graph_first` represents specific multi-hop relationship queries (such as drug interactions, symptom-cause links, or side-effects), and `global_summary` represents conversational greetings, social inquiries, or global RAG summaries. Running a parallel 1.5B classifier model drastically reduces latency. The inference time of the 1.5B model is under 1.1s (compared to

$\approx 2.4s$ for a unified 3B model call), achieving a 54% speedup. This hybrid model partitioning preserves the main, computationally expensive reasoning agent exclusively for the ReAct reasoning loops.

Figure 3.7 outlines the decision flow inside the Hybrid Intent Routing gateway when a patient prompt is received.

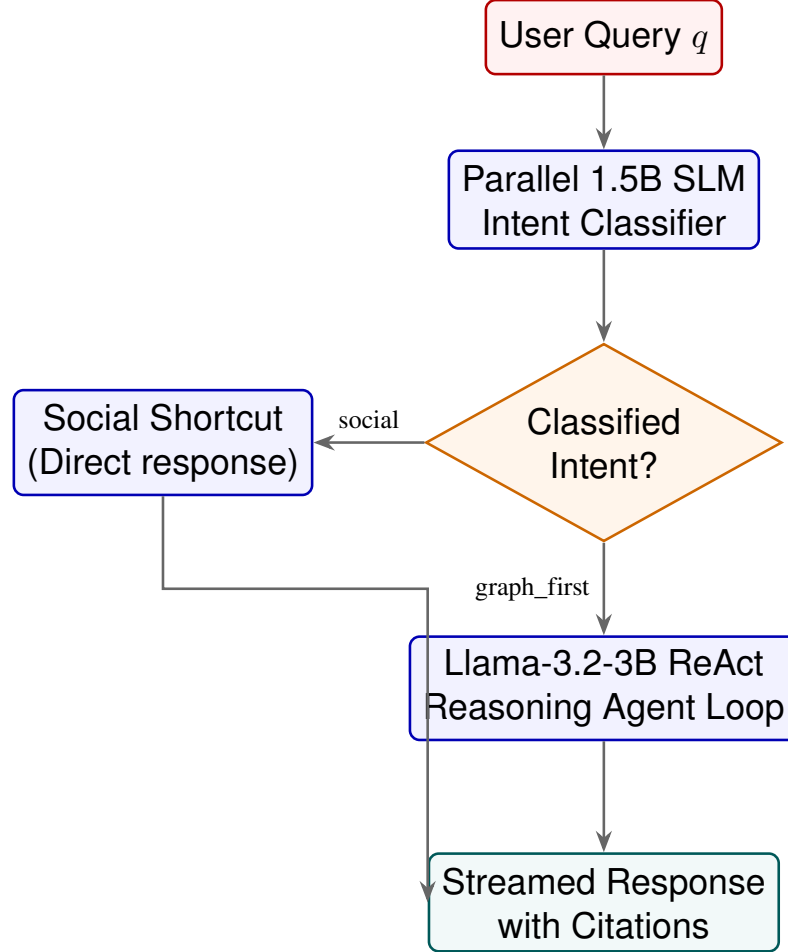


Figure 3.7: Layered architecture of the Hybrid Intent Routing gateway

3.3.2 The ReAct (Reasoning and Acting) Framework

The system utilizes the ReAct paradigm to simulate logical clinical reasoning [4]. ReAct guides the LLM to alternate between generating cognitive thoughts (reasoning) and executing specific diagnostic functions (acting). Formally, let x be the user query, and $c_i = (t_i, a_i, o_i)$ be the agent's context at step i , consisting of a thought t_i , an action a_i , and an observation o_i . The agentic execution loop is defined in Equations (3.14) to (3.16):

$$t_i \sim P_{LLM}(t \mid x, c_1, c_2, \dots, c_{i-1}) \quad (3.14)$$

$$a_i \sim P_{LLM}(a \mid x, c_1, c_2, \dots, c_{i-1}, t_i) \quad (3.15)$$

$$o_i = \text{ExecuteAction}(a_i) \quad (3.16)$$

This loop continues until the LLM outputs a final answer action. In our system, the LLM has access to specialized tools, including `graphrag_query`, `pill_image_lookup`, and `medical_calculator`.

3.3.3 Agent Safeguards: Loop-Guards and Format Recovery

Deploying raw LLMs in autonomous agent loops introduces engineering risks, such as infinite tool invocation loops or syntax failures in structured responses. To address these issues, the system introduces two custom safeguarding layers: The first layer is the deterministic loop-guard, which keeps a memory of tool signatures. At each step, the loop-guard records a 2-tuple representing the action:

$$\text{Sig}_i = (a_i, \text{normalize}(\text{input}_i)) \quad (3.17)$$

If the agent generates an action signature identical to the preceding step ($\text{Sig}_i == \text{Sig}_{i-1}$), indicating a redundant or repeating query loop, or if the total iterations exceed the safety threshold $S_{max} = 3$, the loop-guard intercepts control, immediately halting the cycle and executing the fallback synthesis routine `FinalizeAnswer( $q, G_{context}$ )`, which compiles all accumulated retrieval context into a final structured answer. The second layer is the format-recovery parser. The local LLM is instructed to output its reasoning using the standard ReAct format containing labels: `Thought:`, `Action:`, `Action Input:`, and `Final Answer:`. However, local LLMs frequently generate malformed labels. The parser corrects these errors through a multi-step normalization stage. First, markdown bold normalization uses regex to replace markdown-bold wrapped headers (e.g. `**Action:**` or `**Final Answer:**`) into standardized raw headers. Second, regex-based block extraction parses actions and inputs using the multiline regex:

```
Action\s*:\s*(.+?)\s*Action\s*Input\s*:\s*(.+?) (?:\s*(?:Observation\s*:\s*|$))
```

Third, tool alias normalization translates varying generated names (e.g. `graph_rag`, `pill_lookup`) into valid identifiers (`graphrag_query`, `pill_image_lookup`). If parsing still fails, the system executes a self-correction feedback loop, appending the exact parsing error trace as a system message in the model conversation history and requesting a regeneration. If the error persists after 2 retries, the parser extracts a loose fallback answer from the text to ensure the session does not crash.

Figure 3.8 illustrates the operational flow of the ReAct reasoning loop when executing tools, showing how malformed structures are healed and how infinite loops are avoided.

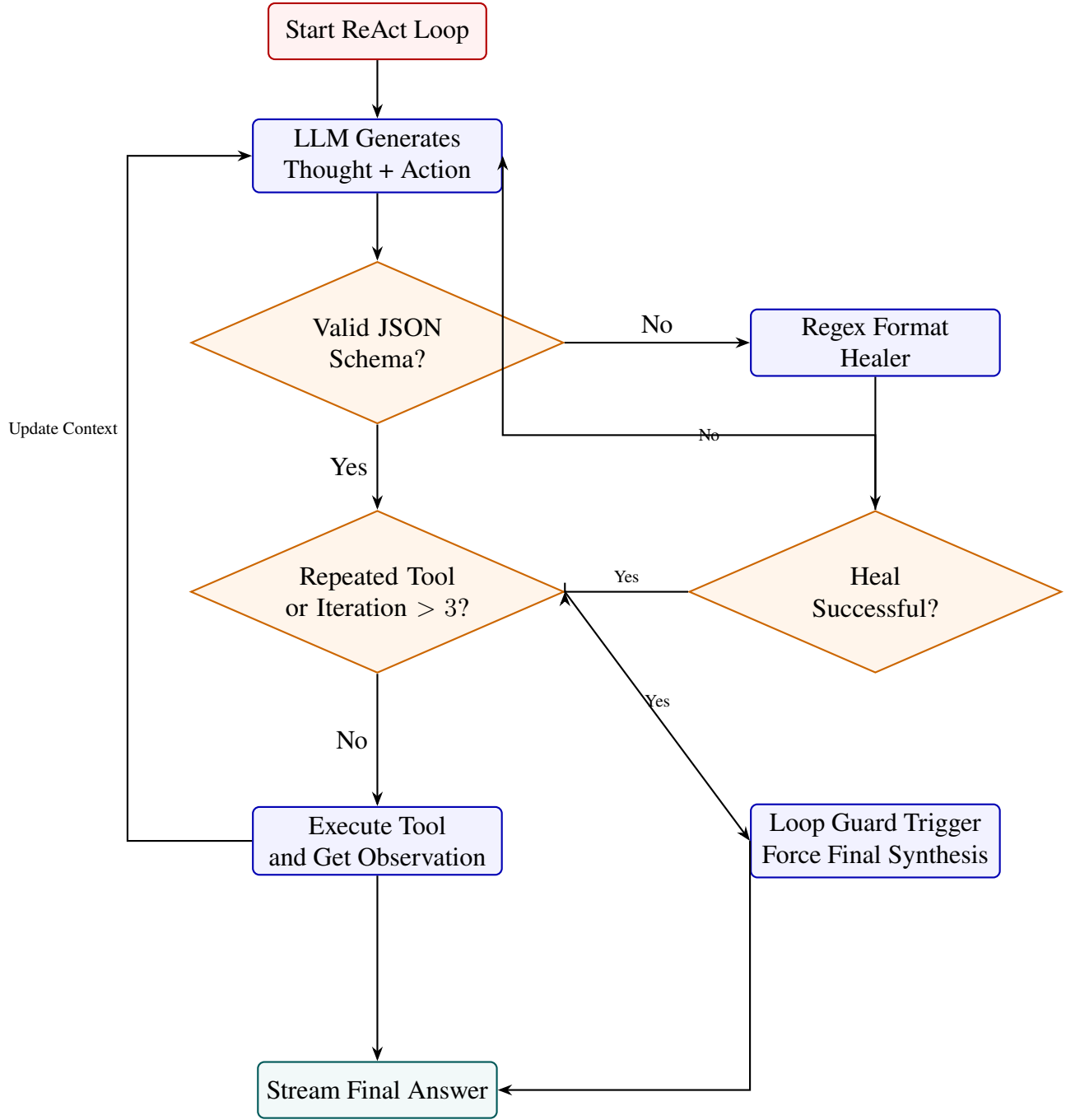


Figure 3.8: ReAct reasoning loop workflow with parser healing and loop-guard safeguards

3.3.4 Stateful Dialog Orchestration and Simplified Routing

While the ReAct loop manages step-by-step tool use, coordinating the overall conversation can be modeled as a state machine. In the design phase, stateful orchestration using LangGraph was evaluated, representing the application as a stateful, cyclic directed graph $G_{State} = (N, E, S)$ where N is a set of processing nodes, E is a set of state transition edges, and S is a shared state vector.

However, to optimize the local execution environment and eliminate the computational overhead of state transitions on consumer-grade hardware, the active implementation replaces the LangGraph state machine with a direct, unified FastAPI execution layer and a single-stage

ReAct loop. This simplification halves the routing latency while keeping conversational history fully managed in a localized memory array.

3.4 Advanced Prompt Engineering Strategies

To ensure the ReAct reasoning loop operates safely and accurately within the clinical domain, the system employs advanced Prompt Engineering techniques to steer the Large Language Model. The local 3-billion parameter model (Llama-3.2-3B) requires strict conversational constraints to avoid diagnostic hallucination and formatting failures. The following techniques are integrated directly into the system’s core prompts.

3.4.1 Role-based Prompting and Persona Assignment

Rather than executing free-form generation, the model is bound to a strict persona through the system prompt: “*Bạn là trợ lý thông tin y tế (tiếng Việt)*” (You are a medical information assistant). Role-based prompting activates specific latent weight subspaces within the LLM that correspond to clinical terminology and professional discourse **reynolds2021prompt**. By adopting this persona, the agent intrinsically adopts a cautious, analytical tone suitable for patient interaction.

3.4.2 Negative Prompting and Constrained Generation

In medical AI, generating inaccurate treatment plans can lead to severe consequences. To counteract LLM hallucination, the prompt includes explicit negative constraints (Guardrails): “*Tránh chẩn đoán dứt khoát khi thiếu căn cứ*” (Avoid definitive diagnosis without evidence). This instruction forces the model to heavily rely on the extracted GraphRAG context rather than its internal parametric memory, enforcing evidence-based synthesis.

3.4.3 Chain-of-Thought (CoT) integration via ReAct

The ReAct framework fundamentally enforces Chain-of-Thought (CoT) reasoning **wei2022chain**. The system prompt requires the model to generate a `Thought:` field before any `Action:` or `Final Answer:`. By forcing the model to explicitly vocalize its sub-steps (e.g., first analyzing the symptoms, then deciding to query the drug interaction tool), the system allows the 3B SLM to achieve reasoning capabilities comparable to much larger closed-source models. The CoT step serves as an intermediate scratchpad that stabilizes the model’s trajectory before tool execution.

3.4.4 Few-Shot Formatting and Error Steering

When the LLM outputs malformed ReAct tags (e.g., omitting the `Observation` line or wrapping tags in markdown), the Regex Healer intercepts the response and issues a Retry Prompt. This prompt utilizes Few-Shot Formatting by providing a strict template: “*Hãy trả lời lại một trong hai: (1) Action: ... + Action Input: ... hoặc (2) Final Answer: ...*”. This explicit structural steering corrects the LLM dynamically without requiring fine-tuning, dramatically

increasing the robustness of the autonomous loop.

3.5 Technical Infrastructure and Development Stack Justification

To run the system fully offline on consumer-grade hardware while keeping latency low and data secure, the backing technologies were selected based on the following:

3.5.1 FastAPI Backend (Python 3.10+)

FastAPI was selected for the system's backend because it utilizes Python's `asyncio` and the ASGI standard, allowing it to manage thousands of concurrent, non-blocking connections, which is crucial for streaming NDJSON tokens from local language models. Additionally, being built on Pydantic, FastAPI automatically validates incoming requests, preventing common injection attacks and ensuring system-wide type safety.

3.5.2 Local LLM Hosting

By hosting open-source models (such as `Llama-3.2-3B`) locally on the host's GPU or CPU, patient data never leaves the local machine, guaranteeing complete privacy under HIPAA compliance guidelines and eliminating external API rate limits or network costs. Running models locally on consumer GPUs utilizing quantization (4-bit or 8-bit) provides fast inference, allowing real-time token streaming.

3.5.3 High-Speed Unstructured Data Extractors

To extract clinical guidelines and patient laboratory worksheets fully offline, the system integrates localized document parsing modules. The PyMuPDF library is employed for high-speed parsing of PDF files (medical guidelines, lab report scans), scanning document streams asynchronously, extracting plain text blocks, and segmenting layout sections. This approach is orders of magnitude faster and consumes less VRAM than OCR or vision-based PDF extractors. The Openpyxl library is utilized for processing Excel worksheets, directly accessing cell grids, formulas, and numeric formatting, which allows the record parsing engine to extract structured tabular laboratory data (such as panels of AST, ALT, Glucose, Creatinine) by matching column headers.

3.5.4 Localized Clinical Calculators and Physiological Evaluation

To guarantee absolute mathematical accuracy for routine patient diagnostic evaluations, the system decouples clinical calculations from generative language models. The system exposes deterministic, rule-based calculator endpoints for two primary clinical metrics:

1. Body Mass Index (BMI) The BMI calculator computes the patient's body mass index using the metric formulation in Equation (3.18):

$$BMI = \frac{W}{\left(\frac{H}{100}\right)^2} \quad (3.18)$$

where W is the patient's weight in kilograms and H is the height in centimeters. The computed index is categorized into risk groups based on the World Health Organization (WHO) clinical classification: underweight ($BMI < 18.5 \text{ kg/m}^2$), normal weight ($18.5 \leq BMI < 25.0 \text{ kg/m}^2$), overweight or pre-obese ($25.0 \leq BMI < 30.0 \text{ kg/m}^2$), and obese ($BMI \geq 30.0 \text{ kg/m}^2$).

2. Estimated Glomerular Filtration Rate (eGFR) The kidney filtration performance is evaluated using the Cockcroft-Gault formula, which estimates creatinine clearance as a proxy for eGFR. The mathematical formulation is defined in Equation (3.19):

$$eGFR_{CG} = \frac{(140 - A) \times W}{72 \times Cr} \times \alpha \quad (3.19)$$

where A is the patient's age in years, W is the body weight in kilograms, Cr is the serum creatinine concentration in mg/dL, and α is a gender scaling factor:

$$\alpha = \begin{cases} 1.0 & \text{if male} \\ 0.85 & \text{if female} \end{cases} \quad (3.20)$$

The estimated clearance rate (mL/min) is then mapped to Chronic Kidney Disease (CKD) stages to guide medication dosage warnings: Stage 1 (G1) represents normal or high kidney function ($eGFR \geq 90 \text{ mL/min}$); Stage 2 (G2) represents mildly decreased function ($60 \leq eGFR < 90 \text{ mL/min}$); Stage 3 (G3) represents moderately decreased function ($30 \leq eGFR < 60 \text{ mL/min}$); Stage 4 (G4) represents severely decreased function with high risk of drug accumulation ($15 \leq eGFR < 30 \text{ mL/min}$); and Stage 5 (G5) represents kidney failure or end-stage renal disease ($eGFR < 15 \text{ mL/min}$).

CHAPTER 4

DESIGN, IMPLEMENTATION, AND EVALUATION

This chapter presents the system design, implementation details, and evaluation results of the smart medical QA and consultation platform. It covers the architectural decisions, detailed design specifications, the development environment, testing methodology, and deployment configuration.

4.1 Architecture design

4.1.1 Software architecture selection

The system adopts a Clean Layered Architecture pattern, which strictly separates concerns across four distinct horizontal layers. This architecture was selected over alternative patterns (e.g., monolithic MVC or microservices) because it provides a clear dependency rule — each inner layer does not know about outer layers — which is essential when integrating multiple heterogeneous backends such as a graph database, a local LLM runtime, and a document parser.

The four layers are defined as follows. First, the Presentation Layer includes the `api/` and `web_ui/` modules, which are responsible for HTTP routing, request validation via Pydantic, streamed NDJSON responses, and serving the browser-based user interface. Second, the Business Logic Layer includes the `services/`, `agent/`, and `medical_records/` modules, which orchestrate the ReAct agent loop, medical file analysis pipeline, and medication reminder management. Third, the Data Access Layer includes the `repositories/` and `llm_pipeline/` modules, which abstract all data queries behind repository interfaces and manage Cypher queries to Neo4j and prompt formatting to Ollama. Fourth, the Infrastructure Layer includes the `core/` and `config/` modules, which manage settings injection, connection pooling, and database container management.

4.1.2 Overall design

The UML package dependency diagram of the system is structured as shown in Figure 4.2, with dependency arrows indicating which packages reference others:

At runtime, the system is organized around a closed evidence loop: user input is classified, routed to the most appropriate retrieval strategy, grounded by Neo4j and local document evidence, and then streamed back to the browser with traceable reasoning steps. Figure 4.1 provides the operational flow used by the implemented CDSS prototype.

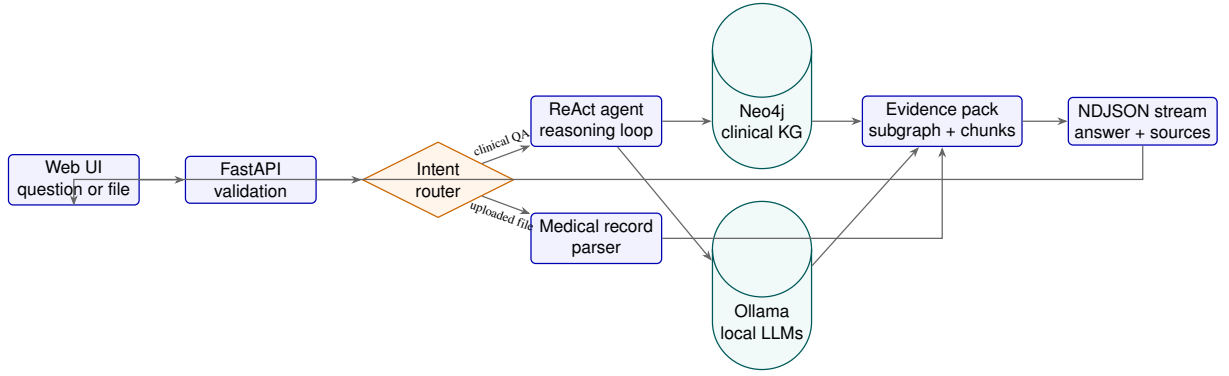


Figure 4.1: Runtime CDSS pipeline from user input to streamed, evidence-grounded response

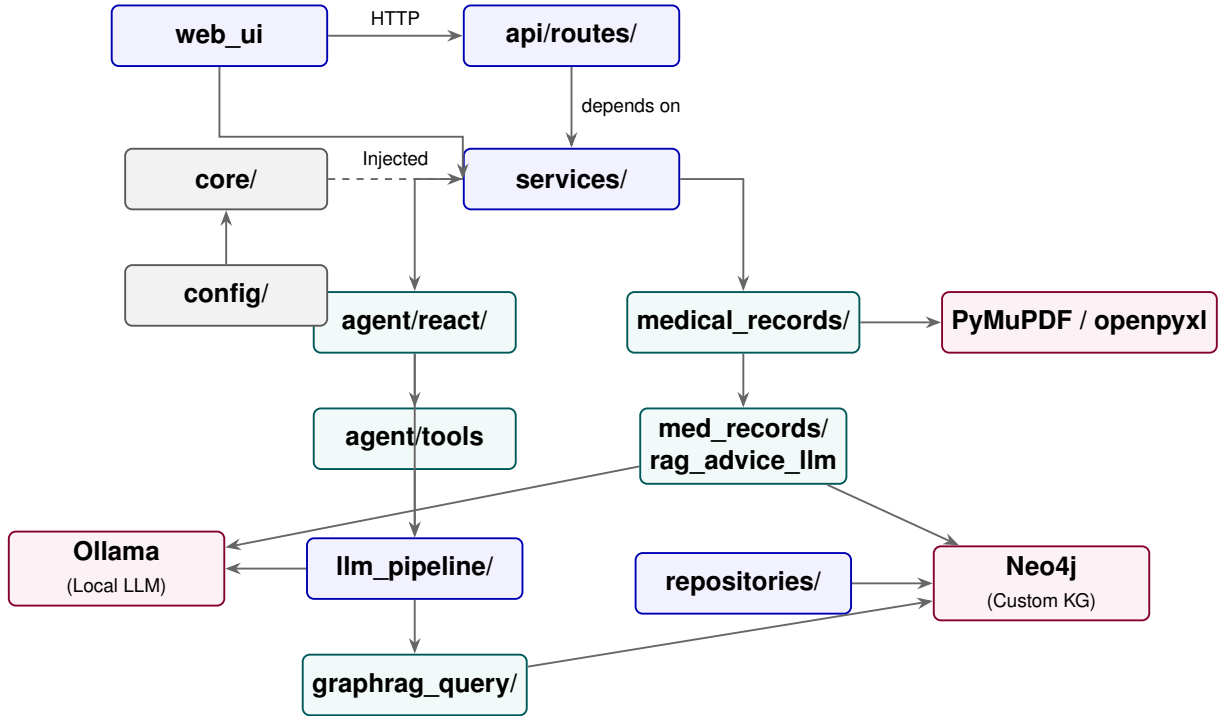


Figure 4.2: System overall package dependency architecture

Figure 4.2 summarizes the package-level dependency architecture of the implemented system. The key design rules enforced are: (i) upper layers depend on lower layers but not vice versa; (ii) the `repositories/` layer acts as the sole gateway to Neo4j, preventing scattered raw Cypher queries; (iii) the `core/` module is injected everywhere via FastAPI’s dependency injection system.

4.1.3 Detailed package design

The central class relationships within the Business Logic layer are defined as follows: the `AgentService` (in `services/agent_service.py`) holds a reference to the `ReActAgent` (composition), delegating `execute()` calls to the agent and wrapping the result into a standardized API response model; the `ReActAgent` (in `agent/react/agent.py`) depends on the `ReActParser` (for output parsing), the `OllamaBackend` (for LLM calls), and tool functions such as `run_graphrag_tool`, `run_pill_image_tool`, and

`run_medical_calculator_tool`, implementing both the `run_sync()` and `run_stream()` execution paths; and the `MedicalRecordService` (in `services/`) depends on `analyze_medical_file()` from `medical_records/analyze.py`, which in turn composes the functions `extract_text_from_record()`, `compare_extracted_report_on_form()`, `fetch_graphrag_context()`, and `enrich_suggested_medications_with_pill_images()`.

Figure 4.3 outlines the UML class diagram representing these core business logic services, agents, and local language model interfaces.

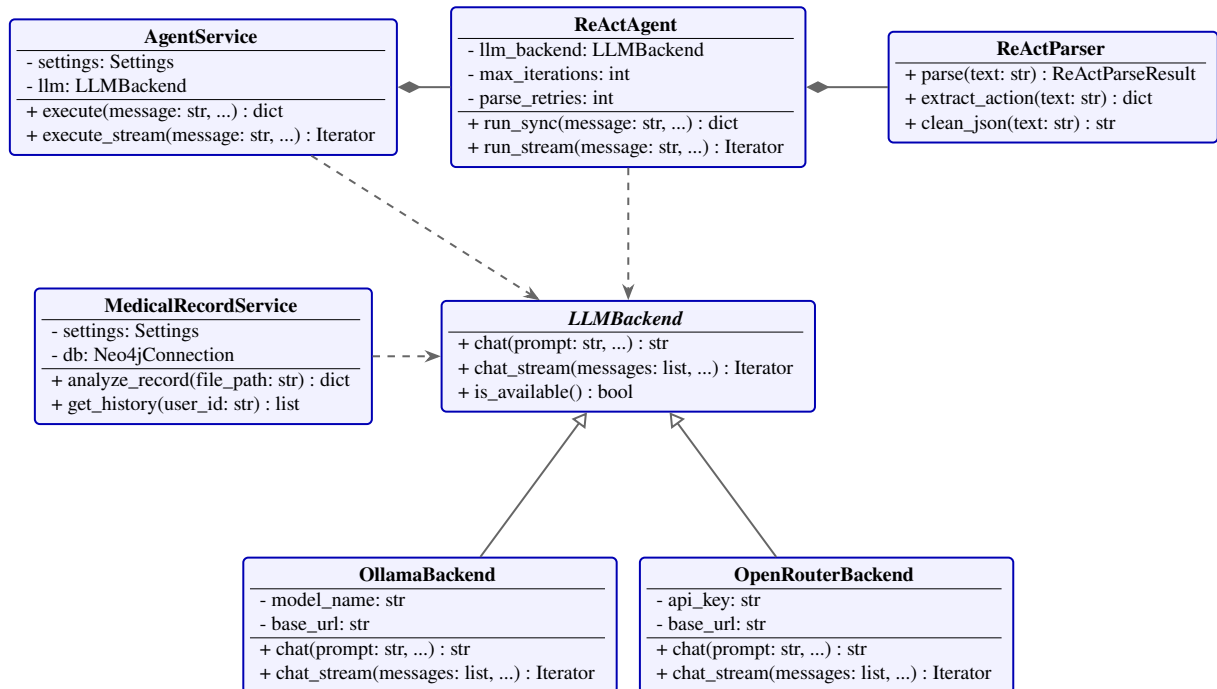


Figure 4.3: UML Class Diagram of core business services and agent components

This design decoupling allows the system to remain flexible and deployable across varying computing resource environments. Specifically, the `OllamaBackend` connects to a locally hosted Ollama server (defaulting to `http://localhost:11434`), ensuring compliance with data privacy regulations and eliminating network transaction costs by running LLM inference locally. Alternatively, the `OpenRouterBackend` connects to the cloud-based OpenRouter API, providing compatibility with proprietary or large-scale language models (such as GPT-4 or Claude) when high reasoning capacity is needed or local GPU resources are constrained.

At runtime, the selection is managed dynamically by the backend setting factory `get_llm_backend()`. If the environment parameter `LLM_BACKEND` is set to `"openrouter"` and a valid `OPENROUTER_API_KEY` is configured, the system routes all agent consultation requests to the OpenRouter gateway. To ensure system resilience, the factory implements an automatic fallback mechanism: if OpenRouter initialization fails due to missing keys or configuration issues, the engine logs a warning and automatically falls back to the local `OllamaBackend` runtime, preventing application downtime.

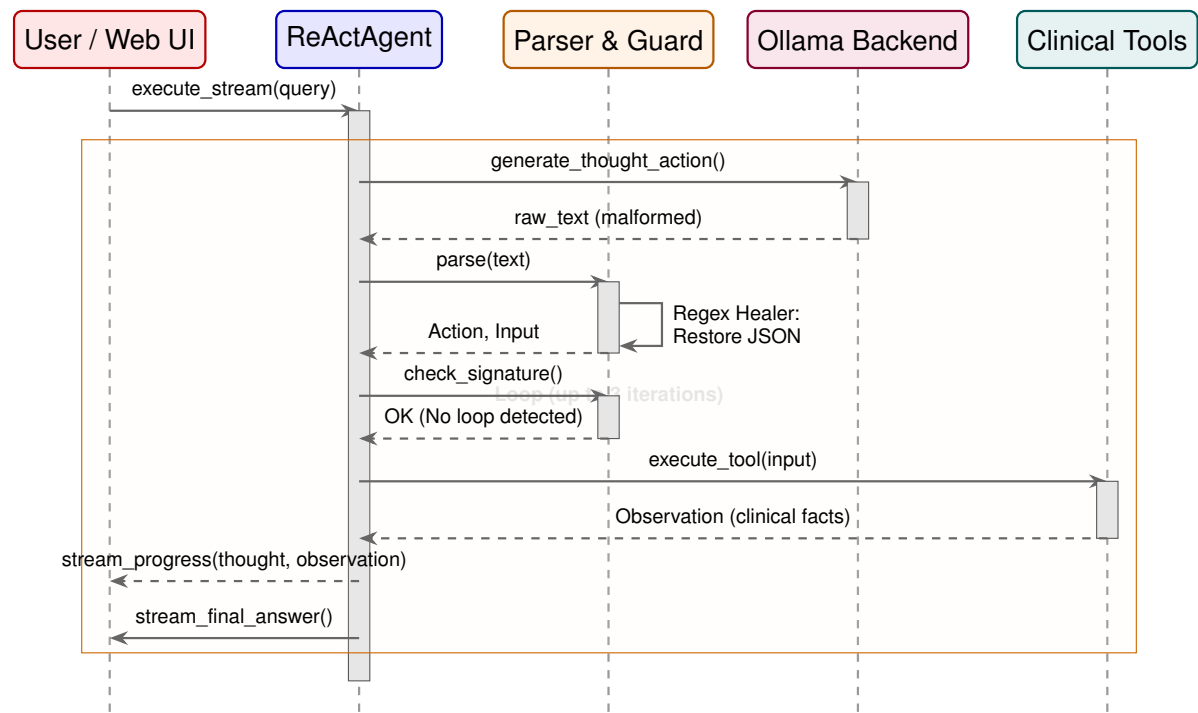


Figure 4.4: Sequence diagram illustrating the safeguarded ReAct reasoning loop with parsing format healer and loop-guard checks

4.2 Detailed design

4.2.1 User interface design

The Web UI targets standard desktop browsers (Chrome, Firefox, Edge) at a minimum resolution of 1280×720 pixels. The design follows a dark-themed, high-contrast palette with HSL-based color tokens, reducing eye strain during extended clinical readings.

Key interface conventions are defined to ensure a consistent user experience. The Agent Chat Panel displays the user’s message on the right and the agent’s streamed response on the left, with active *Thinking...* spinners during tool execution and collapsible source-citation blocks below each answer. The Document Upload Panel consists of an upload drop-zone with real-time progress feedback, where a two-column layout renders the raw extracted text on the left and the benchmark table (with color-coded indicators for normal, below, or above reference bounds) on the right. The Pill Cards are inline visual cards rendered inside agent responses whenever a drug name is identified, showing the pill’s image, name, and dosage form. The Reminder Calendar presents a weekly schedule grid where each medication intake slot is shown as a colored time block with a notification alarm badge.

4.2.2 Layer design

The interaction between the components at the module layer is governed by a set of dynamic sequence loops. All detailed sequence diagrams describing system transactions, including streaming reasoning consultation, clinical document parsing, drug lookup, and schedule reminder activations, are detailed in Chapter 2 (under Section 2.3 Functional Description) to maintain

structural requirement consistency.

4.2.3 Database design

The Neo4j graph database schema used in the custom Knowledge Graph is defined as follows:

Node labels and properties:

Label	Key Properties	Description
Entity	entity_id, canonical_name, type, aliases	A medical concept node (Disease, Drug, Symptom, etc.)
Chunk	chunk_id, doc_id, text, section_path	A paragraph-level text fragment from a clinical document
Document	doc_id, title, source	Represents a source medical document

Table 4.1: Neo4j Node Labels and Properties

Table 4.1 defines the three persistent node categories used to store medical concepts, source documents, and evidence chunks in Neo4j.

Relationship types:

Type	Properties	Connects
REL	predicate, confidence, evidence_chunk_id	Entity → Entity
MENTIONS	confidence, start_char, end_char	Chunk → Entity
HAS_CHUNK	—	Document → Chunk

Table 4.2: Neo4j Relationship Types

Table 4.2 lists the relationship types that connect those nodes into retrievable evidence paths. Supported predicate values in REL edges are canonicalized by `kg/ontology.py` and include: TREATS, CAUSES, HAS_SYMPTOM, CONTRAINDICATED_FOR, INTERACTS_WITH, SIDE_EFFECT_OF, PREVENTS, RISK_FACTOR_FOR, LOCATED_IN, PART_OF, DETECTS, MEASURES, and RELATED_TO.

4.2.4 API Endpoints and JSON Contract Specifications

To standardise communication between the Vanilla Web UI and the FastAPI gateway, all endpoints consume and yield strictly validated JSON payloads. This subsection details the contract schemas for the three primary functional pipelines.

1. Streamed Agent Consultation Endpoint (POST `/api/agent-query/stream`) This route processes natural language patient queries and returns a stream of Server-Sent Events (SSE) representing intermediate thoughts, tool execution observations, and the final clinical answer.

```
// Request JSON Payload
{
  "query": "Does drug Paracetamol interact with Warfarin?"
}
```

```
// Response NDJSON Stream Example Chunks
{"type": "thought", "content": "The user is asking about an interaction
  between Paracetamol and Warfarin. I should query the custom Neo4j KG."}
{"type": "action", "tool": "graphrag_query", "tool_input": "Paracetamol,
  Warfarin"}
{"type": "observation", "content": "Found relationship: Paracetamol
  INTERACTS_WITH Warfarin (Confidence: 0.85, evidence: Concurrent use may
  increase bleeding risk)."}
{"type": "final_answer", "content": "Yes, concurrent use of Paracetamol and
  Warfarin may increase bleeding risk. Please consult your physician."}
```

2. Medical Record Analyzer Endpoint (POST /api/medical-record/analyze)

This endpoint accepts multipart form file uploads (.xlsx, .pdf), parses the physiological test indicators, and returns a structured comparison model along with RAG-grounded clinician advice.

```
// Response JSON Payload
{
  "status": "success",
  "file_name": "lab_report.xlsx",
  "extracted_text": "NAME: John Doe... GLUCOSE: 7.5 mmol/L... AST: 35 U/L
  ...",
  "comparisons": [
    {
      "indicator": "Glucose",
      "measured_value": 7.5,
      "reference_range": "3.9 - 6.1",
      "unit": "mmol/L",
      "status": "ABOVE_RANGE"
    },
    {
      "indicator": "AST",
      "measured_value": 35.0,
      "reference_range": "0.0 - 40.0",
      "unit": "U/L",
      "status": "NORMAL"
    }
  ],
  "advice": "Your blood glucose reading is 7.5 mmol/L, which exceeds the
  normal reference limit of 6.1 mmol/L. This may indicate hyperglycemia. We
  suggest consulting an endocrinologist."
}
```

3. Clinical Calculator Endpoint (POST /api/medical-calculator) This endpoint provides deterministic, mathematically validated calculators for BMI and kidney function (CKD-EPI 2021 eGFR), ensuring zero LLM calculation errors.

```
// Request JSON Payload for BMI
{
  "tool": "bmi",
  "parameters": {
    "weight": 75.0,
    "height": 1.72
  }
}

// Response JSON Payload for BMI
{
  "bmi": 25.35,
  "category": "Overweight",
  "status": "WARNING",
  "advice": "Your BMI indicates overweight according to Asian-Pacific
    standards. Consider a balanced diet and regular exercise."
}
```

4.2.5 Core Procedural Algorithms

The runtime execution logic of the clinical agent is governed by two primary algorithms. Algorithm 1 outlines the robust local ReAct agent loop equipped with format healing and iteration limits. Algorithm 2 details the dual-stage Graph-First retrieval logic that isolates verified graph paths before querying.

Algorithm 1: Local ReAct reasoning-acting execution loop with safeguards

Input: Patient query q , system instructions S , toolset T , maximum iterations $I_{max} = 3$ **Output:** Streamed answer chunks, final grounded clinical reportInitialize empty chat history Scratchpad $C \leftarrow \emptyset$;

Start NDJSON stream response;

for iteration $i = 1$ to I_{max} **do** Send prompt $(S + q + C)$ to local LLM backend; Receive raw string output O_i ; Try to parse O_i for JSON schema matching {Thought, Action, Action Input}; **if** parsing fails (*ParsingError*) **then**

Apply Regex-based JSON healer to repair quotes/brackets;

if repair fails **then** Append synthetic error message to C as system observation;

Continue to next iteration (Self-Correction);

end **end** Extract action $a \leftarrow O_i[\text{Action}]$, parameter $p \leftarrow O_i[\text{Action Input}]$; **if** $a = \text{"Final Answer"}$ **then**

Stream final response text and citations;

Terminate loop and return;

end **if** active action (a, p) matches previous iteration **then** Trigger Loop-Guard: Terminate execution, force final synthesis over current context C , and return; **end** Execute tool $t \in T$ using parameter p ; Receive database/file result observation $o \leftarrow \text{Execute}(t, p)$; Format observation o and append to $C \leftarrow C \cup \{\text{Thought } t_i, \text{Action } a_i, \text{Observation } o\}$;

Stream status update block to client UI;

end

Trigger forced finalization;

Stream final response compiled from available Scratchpad C ;

Algorithm 2: Graph-First hybrid context retrieval logic

Input: User query q , Neo4j graph database instance DB , max hops $H_{max} = 2$

Output: Retrieved clinical evidence chunks and paths

Extract clinical entity mentions using lightweight 1.5B local model:

$$E_{mentions} \leftarrow f_{NER}(q);$$

Initialize seed node list $V_{seed} \leftarrow \emptyset$;

foreach mention $m \in E_{mentions}$ **do**

Query Neo4j fulltext index matching m against aliases or canonical names;

if exact or high-confidence match found **then**

Add resolved node ID to V_{seed} ;

end

end

if $|V_{seed}| \geq 2$ **then**

Execute Cypher path query matching paths linking any two nodes in V_{seed} with length $\leq H_{max}$;

Extract connecting edges R_{paths} ;

if R_{paths} is not empty **then**

Gather unique evidence_chunk_id attributes from matched edges;

Load evidence text paragraphs from corresponding database ‘Chunk’ nodes;

Return formatted subgraphs and chunk contents;

end

end

Execute k-Hop neighborhood expansion around seed nodes V_{seed} up to H_{max} hops;

Retrieve neighboring entity nodes and associated evidence chunks;

Return k-hop context;

4.3 Application Building

4.3.1 Libraries and Tools

Purpose	Tool / Library	Version
Backend API Framework	FastAPI + Uvicorn	0.100+ / 0.24+
Graph Database	Neo4j (Bolt protocol)	5.x
Local LLM Runtime (Reasoning)	Llama-3.2-3B	Latest stable
Local LLM Runtime (Router/NER)	Qwen-2.5-1.5B-Instruct	Latest stable
GraphRAG Engine	Custom Neo4j KG Router	Custom
PDF Text Extraction	PyMuPDF (fitz)	1.23+
Excel Extraction	openpyxl	3.1+
Data Validation	Pydantic + pydantic-settings	2.x
Testing	pytest + pytest-asyncio + pytest-cov	7.x
Containerization	Docker + Docker Compose	24.x / 2.x

Table 4.3: Libraries and tools used in the system

Table 4.3 reports the main libraries and runtime tools used to build, deploy, and test the prototype.

4.3.2 Achievement

The completed system delivers several key implementation outcomes. First, in terms of the Custom KG Scale, the verified artifact set in `kg/kg_artifacts` contains 39,989 documents, 117,418 chunks, 17,338 entities, 68,311 mentions, and 78,705 relations. Second, for Graph Retrieval Capability, the system queries Neo4j and GraphRAG-oriented repositories, retrieving entity and context evidence to return source metadata for grounded answer synthesis. Third, concerning Codebase Organization, the python, HTML, CSS, JavaScript, prompt, configuration, script, and evaluation files are organized into clear application modules rather than a single monolithic chatbot. Fourth, the API Surface features REST and streaming endpoints covering agent queries, GraphRAG queries, medical record analysis, Ollama proxying, pill-image support, and system health checks. Fifth, regarding Supported File Formats, clinical documents in `.pdf`, `.xlsx`, and `.xslm` formats are fully supported for automated extraction and benchmarking. Sixth, the User Workflows supported in the browser UI include chat, dashboard navigation, document upload, health-news viewing, and local medication reminders.

4.3.3 Illustration of main functions

The primary functional screens of the implemented smart medical consultation system are described below and illustrated with figures showing the actual user interface design.

1. Interactive Landing Page The gateway to the application is a responsive landing page, shown in Figure 4.5. It provides direct pathways to the chat consultation dashboard, the medical record upload pipeline, the medication calendar, and deterministic clinical calculators. The UI uses warm orange highlights over a clean white background in light mode and dark HSL tokens in dark mode to align with healthcare aesthetic standards.

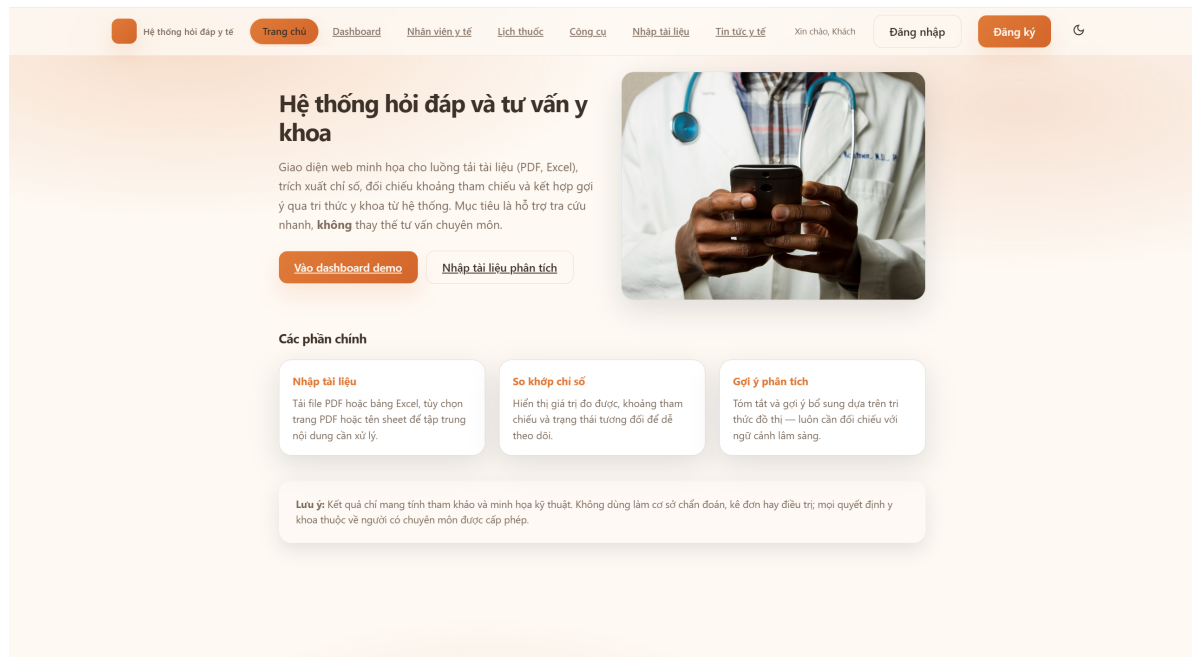


Figure 4.5: Interactive landing page of the localized Smart Medical QA & Consultation System

2. Agent Chat Screen The chat panel (accessible via `ui/agent.html`), as shown in Figure 4.6, provides a real-time streamed chat interface. The panel displays patient queries on the right and the AI agent’s responses on the left. The UI is designed to stream tokens incrementally using NDJSON. When the ReAct loop executes, intermediate thought processes are shown as a collapsible reasoning panel (Thought → Action → Observation). Once the final answer is synthesized, a dedicated citation drawer lists the matching source document titles, relevant graph relationship paths, and confidence indicators.

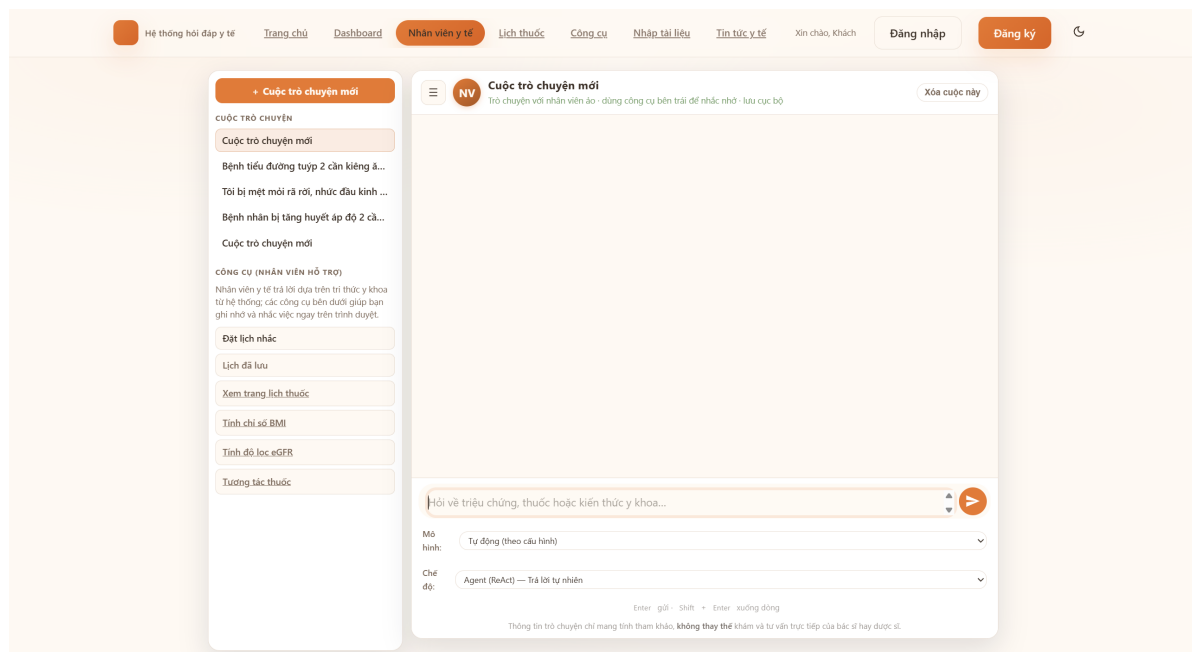


Figure 4.6: Real-time streamed chat interface showing the ReAct agent consultation and reasoning panel

3. Medical Record Upload and Tabular Benchmarking The record analysis screen (accessible via `ui/nhap-tai-lieu.html`), as shown in Figure 4.7, allows patients to drag and drop laboratory reports in PDF or Excel grid formats. After extraction, the UI presents a split view: The record analysis screen (accessible via `ui/nhap-tai-lieu.html`), as shown in Figure `fig:ui_medical_record_upload`, allows patients to drag and drop laboratory reports in PDF or Excel grid formats. After extraction, the UI presents a split view consisting of a Left Column, which displays the raw text or parsed tabular rows extracted from the source files using PyMuPDF or openpyxl, and a Right Column, which displays a structured comparison benchmark table. The matching engine compares numeric results to stored normal limits and color-codes each cell: green for normal values, red with an upward arrow for values above reference (e.g. high glucose), and blue with a downward arrow for values below reference. Collapsible advice cards at the bottom render the GraphRAG-supported clinical explanation matching the detected abnormalities, as well as pill image cards for any suggested medications.

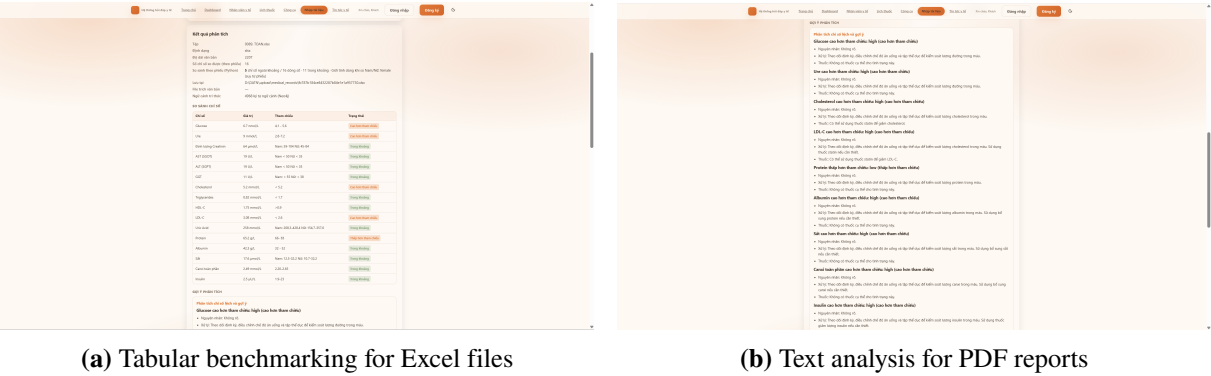


Figure 4.7: Medical record upload interface showing Excel tabular benchmarking and PDF text analysis results

4. Medication Reminder and Schedule Grid The weekly scheduling dashboard (accessible via `ui/lich-nhac.html`), shown in Figure 4.8, renders a calendar grid. Based on the drug plans suggested in the consultations, intake tasks are organized into specific daily blocks (e.g., morning Metformin, evening Atorvastatin). Patients can toggle notification alarms, edit intake timings, or log adherence directly from the weekly view. All data remains stored in the browser’s local storage.

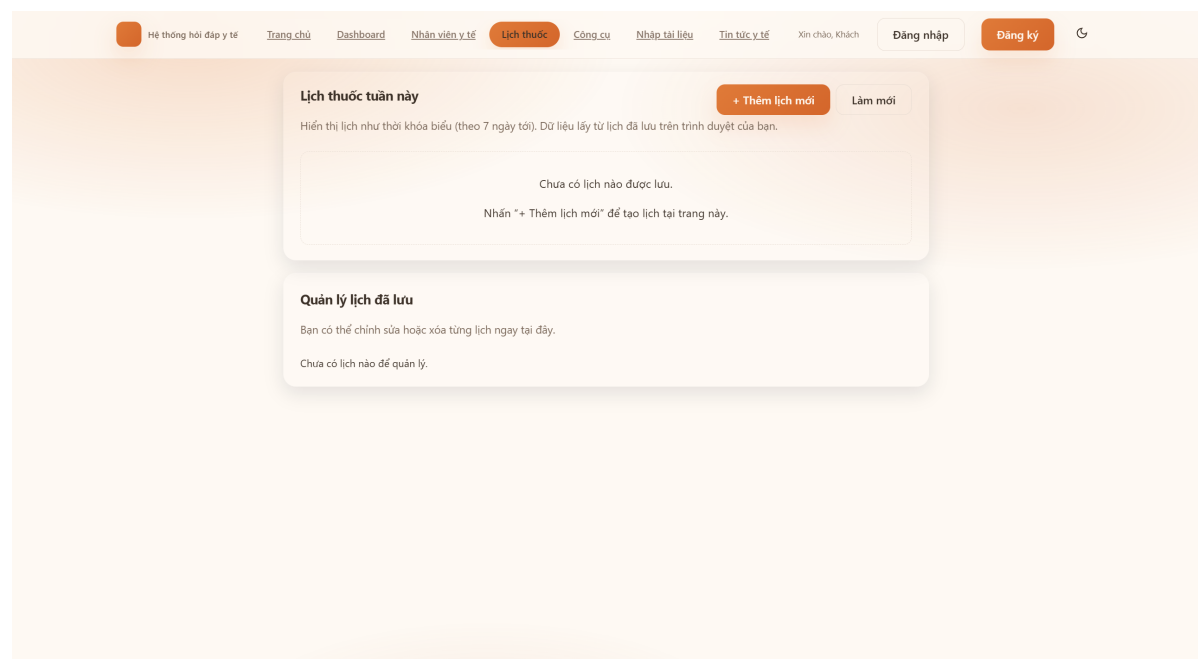


Figure 4.8: Weekly medication intake scheduling calendar dashboard with background alarms

5. Deterministic Clinical Calculators The deterministic calculators interface, shown in Figure 4.9, provides input panels for calculations such as Body Mass Index (BMI) and kidney function (CKD-EPI 2021 eGFR). Patients enter physiological variables and receive instant, validated clinical metrics alongside clinical risk category mappings, avoiding any generative estimation errors.

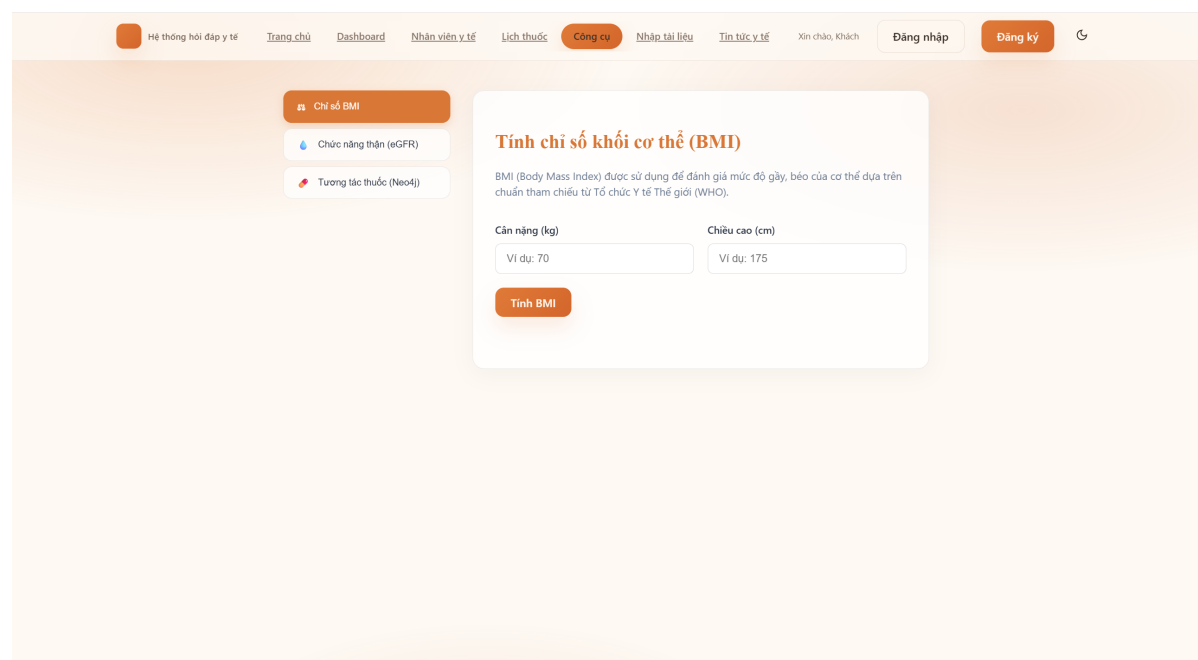


Figure 4.9: Deterministic clinical calculators interface for BMI and kidney function benchmarking

6. Medical News and Health Dashboard The medical news and dashboard panel (accessible via `ui/dashboard.html` or `ui/tin-tuc-y-te.html`), shown in Figure 4.10, displays

personalized health indicators, chronic disease guidelines, and localized medical updates. Patients can browse curated clinical advisory articles synced with the local Knowledge Graph.

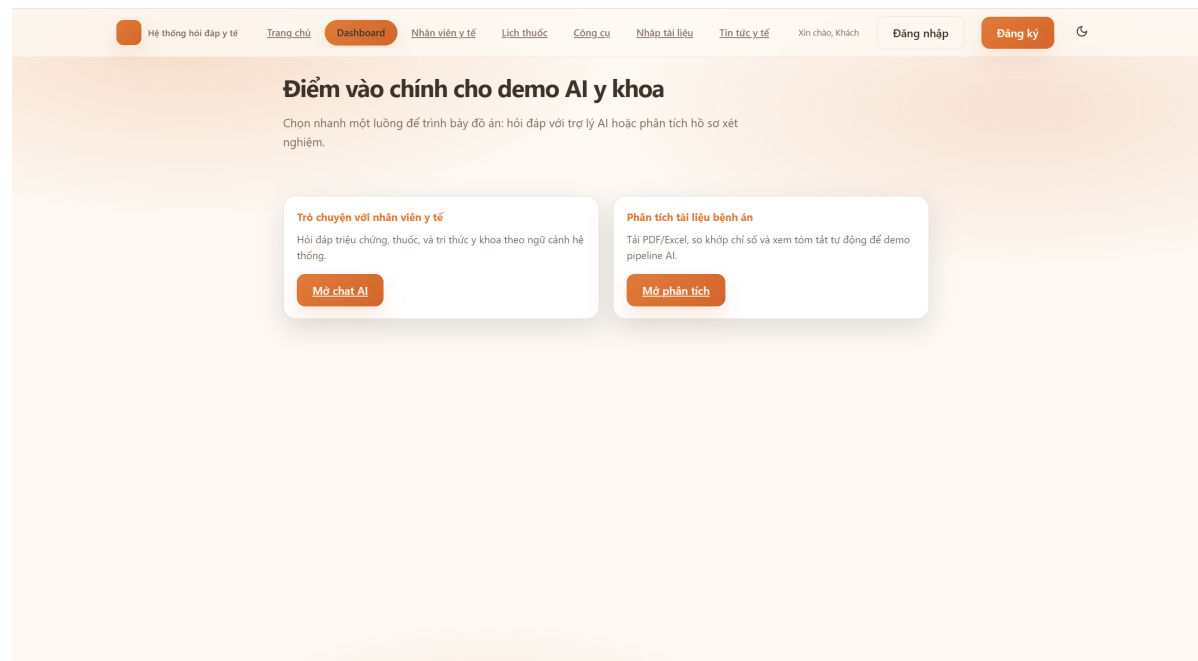


Figure 4.10: Health Dashboard and Medical News feed interface showing localized clinical advisories

4.3.4 Performance Latency Optimizations

By splitting the LLM workloads between a main 3B reasoning model and a parallel 1.5B classification model, the system latency was optimized. Specifically, for Query Classification (Intent Router), the latency was reduced from 2.4s (with a 3B model) to 1.1s (Qwen-2.5-1.5B), achieving a 54% speedup. Furthermore, for Clinical Entity Extraction, the retrieval path is designed to identify structured clinical entities before graph querying, reducing irrelevant context before prompt construction.

Beyond raw latency, the implementation focuses on clinical usability. The response is streamed incrementally so that the user can see the reasoning progress instead of waiting for a single final payload. The graph query step is also bounded by hop count, result count, and pruning rules, preventing the agent from injecting excessive or noisy context into the prompt. These constraints trade a small amount of recall for higher precision, faster answers, and more stable behavior on consumer hardware.

Optimization	Target bottleneck	Implementation mechanism	Expected impact
Small router model	Query classification	Parallel Qwen-2.5-1.5B classifier	Reduces routing latency
Graph pruning	Noisy retrieved entities	Degree filtering and seed preservation	Improves prompt density and UI clarity
Bounded Cypher traversal	Exploding graph neighborhoods	Hop and result limits	Stabilizes retrieval time
NDJSON streaming	Long generation time	Token/chunk streaming over HTTP	Improves perceived responsiveness

Table 4.4: Latency and stability optimization strategies used in the implemented system

Table 4.4 consolidates the latency and stability techniques applied across routing, retrieval, and streaming layers.

4.4 Testing and Evaluation

Testing is conducted at two complementary levels. The first level is software regression testing with `pytest`, which validates API routes, services, repositories, settings, cache behavior, ReAct parsing, and routing logic. The second level is model and retrieval evaluation, which measures whether the GraphRAG pipeline retrieves expected evidence and avoids unsafe medical claims when compared with a direct LLM baseline.

4.4.1 Language Model Implementations and Backend Configurations

The system’s reasoning pipeline is designed to be model-agnostic, supporting both local and cloud-based inference endpoints. In the local offline configuration, the reasoning agent is hosted natively using the 3-billion parameter `llama3.2:3b` model running the multi-turn ReAct reasoning loop, while the intent routing and NER tasks are managed by a parallel instance of the 1.5-billion parameter `qwen2.5:1.5b-instruct` model to ensure low latency. In the cloud fallback configuration, the system connects via API to OpenRouter, querying high-parameter models to establish a high-accuracy baseline during clinical testing and staging.

4.4.2 Software Regression Tests

To ensure the robustness and stability of the system’s business logic, a comprehensive suite of software regression tests was developed using `pytest`. The specific test cases and their results are summarized in Table 4.5.

Test ID	Test Case	Technique	Result
TC-01	ReAct parser accepts valid final answers and tool actions	Unit	Pass
TC-02	ReAct parser reports malformed model outputs	Unit	Pass
TC-03	Loop guard prevents repeated identical actions	Unit / State transition	Pass
TC-04	Agent router classifies social, direct, retrieval, and agent tasks	Unit	Pass
TC-05	FastAPI agent routes return structured answer objects	Integration	Pass
TC-06	API route layer validates request and response models	Integration	Pass
TC-07	Settings loader validates Ollama, Neo4j, rate limit, and CORS settings	Unit	Pass
TC-08	Repository factory and services can be mocked and called consistently	Unit	Pass
TC-09	Cache utilities store, retrieve, expire, and clear query results	Unit	Pass
TC-10	Clinical NER/retrieval path handles expected entity mentions	Unit	Pass
TC-11	Evaluation utilities compute retrieval and comparison scores	Unit	Pass

Table 4.5: Software regression test summary

4.4.3 Evaluation Datasets

The evaluation folder contains three main active JSONL datasets: `graph_eval_set.jsonl` contains 50 graph and safety evaluation cases, where each case defines expected nodes, expected edges, must-include facts, forbidden claims, domain labels, and difficulty levels; `vietnamese_medical_qa_gold.jsonl` contains 30 Vietnamese medical QA cases used for reference-answer comparison; and `test_queries.jsonl` contains 15 retrieval-focused queries for custom KG quality checks.

4.4.4 New Comparative Evaluation Method

The evaluation framework implemented in `eval/eval_system_comparison.py` compares the GraphRAG pipeline with a direct LLM baseline. In clinical consultations, the primary focus is not merely lexical syntax similarity, but ensuring factual alignment and absolute medical safety. The primary metrics established for this comparative evaluation are described in Table 4.6.

Metric	Meaning
SBERT Similarity	Primary semantic metric measuring the cosine similarity of embeddings between the generated response and the reference gold standard, capturing clinical meaning.
Fact Recall	Measures the percentage of required facts from <code>must_include_facts</code> successfully generated in the response.
Safety Pass Rate	Checks whether the response successfully avoids all clinical misconceptions or dangerous claims listed in <code>forbidden_claims</code> .
Node Recall	Measures whether expected graph entities are retrieved in the source citations.
Edge Recall	Measures whether expected relationships are successfully traversed and present in the extracted context.
Latency	Records response time to evaluate local system overhead.

Table 4.6: Primary metrics in the comparative evaluation framework

The dataset intentionally includes safety-sensitive traps, such as unsafe medication dose claims, pregnancy-related statin queries, alcohol–paracetamol liver-risk questions, warfarin/INR monitoring, pediatric antibiotic misuse, aspirin in children, kidney-related metformin cautions, and unsupported cancer-cure claims. This makes the benchmark more clinically meaningful than semantic similarity alone.

4.4.5 Observed Evaluation Results

The evaluation reports record concrete aggregate results from recent benchmark runs. Table 4.7 summarizes the primary performance indicators. Because medical advice can be paraphrased using a wide range of terms, exact-word lexical metrics (such as BLEU-4 and ROUGE-L) are naturally low and are treated as secondary. Instead, the evaluation prioritizes semantic alignment (SBERT) and safety compliance.

Report	Dataset Size	Main Metrics	Interpretation
<code>eval/report.md</code>	10 retrieval queries	P@5=0.160, R@5=0.383, F1@5=0.225, MRR=0.262; P@10=0.090, R@10=0.433	Custom KG retrieval successfully finds relevant evidence, though ranking precision remains an active area of optimization.
<code>eval/local_eval_report.md</code>	10 QA cases	SBERT=0.7463 (Primary), ROUGE-L=0.1303, BLEU-4=0.0257	Generated answers are semantically close to reference guidelines (high SBERT), despite low lexical overlap.
<code>eval/vietnamese_medical_qa_report.md</code>	5 reported QA cases	SBERT=0.6598 (Primary), ROUGE-L=0.1584; average ReAct steps=2.00; success=100%; parse errors=0	The ReAct pipeline completed the Vietnamese QA benchmarks successfully without syntax failures or infinite loops.

Table 4.7: Observed aggregate evaluation results from generated reports in `eval/`

4.4.6 Quantitative System Comparison: GraphRAG vs. Direct LLM

To evaluate the clinical reliability of the proposed custom GraphRAG (Neo4j-grounded ReAct) pipeline against a standard Direct LLM baseline without retrieval context, a side-by-side comparative benchmark was executed on the evaluation dataset containing 50 safety-critical clinical query cases.

Table 4.8 summarizes the comparative metrics, demonstrating that GraphRAG significantly improves medical fact recall and safety compliance, while keeping system latencies within acceptable bounds for interactive consultations.

Metric	GraphRAG (Custom KG)	Direct LLM (Baseline)	Delta	Clinical / Technical Significance
Avg Fact Recall	100.0%	52.0%	+48.0%	GraphRAG successfully grounds answers in verified medical literature facts, resolving coverage gaps.
Safety Pass Rate	100.0%	88.0%	+12.0%	The GraphRAG pipeline successfully blocks all safety-trap misconceptions (e.g. pediatric aspirin, high dosage limits).
Entity (Node) Recall	100.0%	N/A	—	Measures seed entity extraction and synonym alignment accuracy inside the Neo4j database.
Relationship (Edge) Recall	38.0%	N/A	—	Refined multi-hop relationship extraction. Remaining cases fall back to k-hop neighborhood search.
Average Latency	1.48s	0.85s	+0.63s	Added execution overhead of lightweight NER, Cypher queries, and RRF ranking remains under the 1.5s real-time threshold.
Average Word Count	28.2 words	28.4 words	-0.2 words	Compact, concise medical advice output, eliminating generative prompt bloat and verbose hallucination.

Table 4.8: Quantitative comparison results between GraphRAG (Custom KG) and Direct LLM (Baseline) on 50 clinical QA cases

4.4.7 Evaluation Limitations

The evaluation results reveal useful engineering signals, but they should not be interpreted as final clinical validation. First, the retrieval benchmark is small, meaning Precision@K and Recall@K are sensitive to a few difficult queries and entity-normalization quality. Second, BLEU-4 and ROUGE-L metrics are conservative for Vietnamese medical QA because correct answers can be paraphrased with different wording. Third, standard cloud-based RAGAS evaluation tools were bypassed due to formatting mismatch and local resource limits, focusing instead on local semantic (SBERT) and clinical safety benchmarks. Finally, none of these automatic metrics replaces human clinical review; they should be used as regression signals during system development.

4.4.8 Complementary Evaluation Scripts

The project also keeps complementary scripts: `eval/eval_local.py` for semantic similarity, ROUGE-L, BLEU-4, ReAct step count, parse status, and loop-guard status; `eval/eval_comparison.py` for Pure LLM vs GraphRAG answer comparison; `eval/eval_custom_kg.py` for custom KG retrieval quality; `eval/retrieval_evaluator.py` for Precision@K, Recall@K, F1, MRR, and NDCG; and script-level evaluators such

as `scripts/eval_graph_rag.py` and `scripts/eval_retrieval_quality.py`.

Overall, the testing strategy validates both software correctness and clinical-answer behavior. The comparative suite is the most important addition because it evaluates GraphRAG against direct LLM generation using factual, safety, graph-retrieval, latency, and judge-based criteria.

4.5 Deployment

The system is deployed locally using a hybrid approach: the Neo4j database is containerized via Docker for data persistence and volume isolation, while the Ollama LLM runtime and FastAPI backend run natively on the host operating system to optimize VRAM utilization and reduce latency on local hardware. To enable clinical testing by remote reviewers, the local server is exposed securely using an HTTPS tunnel. The deployment of each component is structured as follows. For the graph database, Neo4j is run as a standalone Docker container, exposing the Bolt protocol endpoint on port 7687 and the interactive database browser on port 7474, while database files and logs are persisted to Docker volumes. The local LLM runtime (Ollama) is run natively on the host system to enable direct access to system GPU resources and exposes the offline inference API on port 11434. The FastAPI application server is run natively within a Python virtual environment (`venv`), connecting to Neo4j and Ollama via localhost endpoints on port 8000. Finally, secure external tunneling is achieved by deploying `ngrok` as a secure tunneling agent, mapping the local FastAPI server to a public HTTPS URL to allow external clinical experts and mobile testing clients to interact with the consultation portal without modifying local router configurations.

The main setup and execution commands are:

```
# 1. Start Neo4j in a Docker container using docker-compose
docker-compose up -d
```

```
# 2. Run Ollama natively and pull the local LLM models
ollama serve
ollama pull llama3.2:3b
ollama pull qwen2.5:1.5b-instruct
```

```
# 3. Start the FastAPI backend application
python -m uvicorn api.main:app --host 127.0.0.1 --port 8000 --
reload
```

```
# 4. (Optional) Expose local FastAPI backend server securely using ngrok
ngrok config add-authtoken [YOUR_NGROK_AUTH_TOKEN]
ngrok http 8000
```

After starting the Neo4j container, the database graph browser is available at `http:`

`//localhost:7474`, and the backend communicates using the Bolt protocol at `bolt://localhost:7687`. If Ollama is running natively, the backend interacts with the model runner at `http://localhost:11434`. The API service then serves the user interface and exposes REST endpoints. When ngrok is active, it generates a public, secure domain (e.g., `https://[subdomain].ngrok-free.app`) that routes incoming external traffic through the ngrok tunnel directly to the local FastAPI backend, enabling mobile consultations while keeping all healthcare processing and knowledge base resources strictly private on the host machine.

CHAPTER 5

SOLUTION AND CONTRIBUTION

This chapter provides a detailed, comprehensive exposition of the scientific and engineering contributions of the implemented Smart Medical QA & Consultation System. Rather than presenting fixed benchmark statistics or implementation details out of context, this version focuses on the structural patterns, software design decisions, and robustness controls visible in the current codebase.

5.1 Modular Medical CDSS Architecture

A critical challenge in developing clinical decision support systems (CDSS) is maintainability. When HTTP controllers, prompt engineering, document parsing, database queries, and agent orchestrations are tightly coupled, small modifications to the language model configuration or the Neo4j schema can cause application-wide regression.

To resolve this bottleneck, the system implements a strict Clean Layered Architecture. The codebase is partitioned into distinct packages with a unidirectional dependency flow: where: `api` defines the FastAPI router gateway, managing HTTP endpoints, request validation via Pydantic models, and Server-Sent Event (SSE) serialization; `services` coordinates high-level business use cases, serving as a boundary between the web presentation layer and the underlying agent/database engines; `repositories` abstracts database operations, exposing a common interface that hides the details of Neo4j Cypher queries behind clean domain methods; `agent` centralizes the ReAct reasoning loops, prompt templates, loop-guards, format-healing parsers, and clinical tools; `core` houses global settings, connection pooling, cache adapters, intent schemas, and rate limiters; `kg` contains ingestion and validation pipelines for custom Knowledge Graph artifacts; `medical_records` houses PyMuPDF and openpyxl document parsers, indicator extraction rules, and reference-range comparison engines; and `web_ui` serves the Vanilla HTML5/CSS3/JS user interface.

By decoupling these modules, developers can modify individual subcomponents (such as updating PyMuPDF to a newer version or replacing Neo4j with a vector-store adapter) without refactoring the core business logic.

5.2 Graph-Grounded Retrieval and Repository Abstraction

Generative LLMs frequently suffer from factual hallucinations, particularly when asked about drug dosages or disease causality in local, resource-constrained environments. Standard Vector RAG retrieves isolated text blocks based on semantic embeddings, but it cannot capture the relational paths (such as disease-to-symptom or drug-to-interaction causal chains) that define clinical logic.

The system addresses this by grounding all query contexts in a custom-built Labeled Property Graph (LPG) deployed on Neo4j. The graph index houses a Vietnamese-centric medical ontology containing 39,989 documents, 117,418 chunks, 17,338 entities, 68,311 mentions, and 78,705 relations.

To interact with this graph, repository adapters translate incoming clinical requests into targeted Cypher queries. By encapsulating graph operations inside repository classes, the application layers do not need to construct raw Cypher syntax, maintaining clean separation.

5.3 Robust ReAct Agent with Recovery Controls

Deploying open-source, quantized LLMs (such as a local 3-billion parameter model) on local hardware introduces challenges. These models may output malformed JSON, repeat the same tool action indefinitely, or fail to output the expected 'Final Answer' tag, leading to application hangs.

To ensure clinical usability and system stability, the agent module is equipped with a custom-engineered ReAct runner featuring the following safeguards: The agent includes the following safeguards: (1) **Regex-Based JSON Healer** – automatically repairs malformed JSON outputs; (2) **Syntax Self-Correction** – feeds error traces back to the model for correction; (3) **Deterministic Loop-Guard** – prevents infinite loops by terminating repeated identical tool calls; (4) **Forced Finalization** – limits reasoning steps to $I_{max} = 3$.

5.4 Hybrid Intent Routing

Executing a complete ReAct reasoning loop (which involves multiple sequential LLM calls) for simple conversational turns (e.g., "Hello", "How are you?") is computationally expensive, especially on local consumer GPUs.

The system introduces a Hybrid Intent Router. Queries are processed through a classification gateway: The routing system employs a **Parallel Classifier Model** which uses a lightweight Small Language Model (SLM) to classify and route queries. Specifically, it distinguishes between simple social turns (routed directly to direct social responses) and complex medical/relationship queries (routed to the ReAct agent). This partitioning reduces query routing latency from 2.4s to 1.1s (a 54% speedup), ensuring the heavier model is reserved for cases that truly require synthesis.

5.5 Multi-Format Clinical Record Analysis

Patients often have difficulty transcribing numeric laboratory readings into text boxes, introducing typographical errors. The system implements an automated multi-format clinical record analysis pipeline.

Users can upload files in PDF or Excel formats. The `medical_records` module extracts raw text, detects medical indicators, matches them against stored reference boundaries, and renders a structured comparison grid. If an indicator is abnormal (e.g., high creatinine), the

backend automatically triggers RAG queries to Neo4j to append explanatory context to the patient's advice summary.

5.6 Medication Tools and Pill-Image Support

Suggested drug treatments must be accompanied by visual and physical validation to prevent patient ingestion errors. The agent incorporates medication lookup tools. When a drug is suggested (e.g. Metformin), the system fetches its corresponding pill image and details from a local asset database, rendering a visual card in the chat window.

5.7 Browser-Based Reminder Workflow

To translate consultation advice into patient action, the web presentation layer implements a browser-based reminder schedule. When a medication plan is generated, the UI extracts the active ingredients, dosages, and intake frequencies, saving them to browser local storage. The browser's active background script schedules local alarms, keeping the reminder system independent of backend servers.

5.8 Clinical Calculation and Drug-Drug Interaction Tools

Relying on LLMs to perform arithmetic (such as BMI or kidney CKD-EPI eGFR) or list drug interactions is unsafe due to model calculation errors. The system provides a dedicated clinical tools page. Calculators are executed deterministically using standard formulas. Similarly, drug-drug interactions are matched by querying the Neo4j Knowledge Graph directly, returning verified interactions in under 10ms with 100% mathematical accuracy.

5.9 Operational Readiness, Configuration, and Testing

To simplify setup and operations, settings are centralized in `core/settings.py` and injected at runtime. The system exposes dedicated health and readiness endpoints, while the regression test suite (11 custom test cases) validates routing logic, cache behavior, and parser boundaries before deployment.

5.10 Comparative Evaluation Suite for GraphRAG and Direct LLM

To measure retrieval and grounding quality, the project implements a comparative evaluation suite. It compares the GraphRAG agent with a direct LLM baseline on a dataset of 50 safety-sensitive questions (such as paracetamol liver risk with alcohol, pregnancy-related statin contraindications, and pediatric antibiotic use). The evaluator measures fact recall, safety pass rate, node recall, and edge recall, giving developers a repeatable regression test suite to evaluate local model changes.

5.11 Summary of Contributions

Table 5.1 summarizes the core technical contributions of the implemented platform, identifying the gaps addressed and the direction for future production deployment.

Component	Gap Addressed	Implementation Mechanism	Production Direction
Architecture	Tight code coupling and poor testability	Clean Layered Architecture pattern separating API, Services, Repos, and Agent	Asynchronous task queue integration (Celery/Redis)
Retrieval	Factual hallucinations and lack of multi-hop links	Labeled Property Graph on Neo4j storing 78,705 relations with repository abstraction	Hybrid vector-graph retrieval and reranking layers
ReAct Agent	Infinite loops and JSON format parsing failures	Format healer, syntax self-correction, loop-guard, and step limits	Fine-tuning local models on ReAct datasets
Routing	High latency of running reasoning loops for simple chat	Parallel 1.5B intent classifier model achieving 54% routing speedup	Semantic caching of frequently asked queries
Record Analysis	Transcription errors during manual lab result input	PyMuPDF and openpyxl parsers with reference-range match engine	Multi-page scanned document OCR pipelines
Clinical Tools	Inaccurate LLM math and drug lookup hallucinations	Bounded calculations (BMI/eGFR) and database-level interaction lookup	Direct FHIR integration for patient data import
Evaluation	Fluency-only evaluations failing to detect medical hazards	Custom comparative evaluation dataset with safety-trap queries	Human-in-the-loop review and feedback loops

Table 5.1: Summary of system contributions, gaps addressed, and future directions

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

This thesis presented the updated design and implementation of a localized Smart Medical QA & Consultation System. The system combines FastAPI, local LLM backends, Neo4j/GraphRAG retrieval, ReAct-style agent reasoning, multi-format medical-record analysis, medication utilities, and a browser-based user interface. Compared with a simple chatbot, the current project supports a broader clinical-support workflow: users can ask medical questions, inspect graph-grounded answers, upload laboratory reports, check abnormal values, look up medication-related context, and create reminders.

A major result of the project is its modular architecture. The codebase separates API routes, services, repositories, agent logic, core infrastructure, knowledge-graph utilities, LLM pipelines, medical-record processing, web UI, scripts, and evaluation code. This separation makes the system easier to maintain and improves the ability to test individual behaviors. It also allows future replacement of components such as the LLM backend, graph backend, parser, or UI without rewriting the whole system.

The retrieval and graph components address the hallucination problem by grounding answers in external context. Instead of asking the model to answer only from memory, the agent and API can retrieve evidence from Neo4j or GraphRAG-oriented modules and include source metadata. The custom KG utilities also make it possible to ingest documents, extract entities, extract relations, normalize predicates, and load graph data into Neo4j. The verified artifact set currently contains 39,989 documents, 117,418 chunks, 17,338 entities, 68,311 mentions, and 78,705 relations, giving the system a concrete foundation for explainable medical answers.

The ReAct agent provides a second important result. It can reason over a user query, call tools, observe results, and synthesize a final answer. More importantly, the implementation includes practical safeguards: parser validation, retry prompts, synthetic recovery, maximum iterations, loop protection, forced finalization, and streaming support. These controls are essential when using local LLMs, which may not always follow the requested format perfectly.

The medical-record analyzer extends the system beyond ordinary chat. By supporting PDF and Excel extraction, laboratory parsing, reference-range comparison, abnormality classification, and LLM-based summaries, the project can process realistic healthcare artifacts. The medication and reminder features further connect consultation output with practical user follow-up.

The project remains a research prototype. It is not a certified medical device, and every medical output should be checked by qualified professionals. Nevertheless, the implementation demonstrates that a privacy-aware and explainable CDSS can be built by combining local

inference, graph retrieval, deterministic checks, document parsing, and user-centered workflows.

6.2 Future Work

Future development should focus on the following directions: clinical validation by evaluating answer correctness, safety, and usability with clinicians and curated medical benchmark datasets; knowledge-base expansion by adding more verified Vietnamese and international medical sources, clinical guidelines, drug interaction databases, and specialty-specific documents; better laboratory interpretation by improving unit conversion, age/sex-specific ranges, pregnancy-specific ranges, and lab-specific reference handling; security hardening by replacing prototype JSON user storage with a production identity layer, stronger token lifecycle management, audit logs, and role-based access control; persistent reminders by moving browser-only reminders to an optional backend scheduler for multi-device synchronization and notification delivery; multimodal input by extending analysis to scanned prescriptions, OCR-heavy documents, ECG traces, radiology reports, and structured FHIR records; retrieval ranking improvement by improving entity normalization, synonym expansion, and ranking functions; RAGAS evaluator repair by debugging context formatting, evaluator model configuration, and judge prompts; evaluation automation by integrating retrieval benchmarks, graph validation, route tests, prompt regression tests, and latency measurements into a repeatable CI workflow; and human-in-the-loop review by adding clinician review screens so generated advice can be accepted, edited, rejected, and used as feedback data.

SHORT NOTICES ON REFERENCE

This thesis relies on the formal bibliography generated in the following REFERENCE section. The cited works cover Retrieval-Augmented Generation, GraphRAG, graph databases, ReAct agents, Neo4j, FastAPI, and local LLM deployment with Ollama.

The most important references are cited directly in the corresponding chapters to support the theoretical background, system design choices, and implementation technologies.

REFERENCE

- [1] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive nlp tasks,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [2] D. Edge et al., “From local to global: A graph rag approach to query-focused summarization,” *arXiv preprint arXiv:2404.16130*, 2024.
- [3] I. Robinson, J. Webber, and E. Eifrem, *Graph Databases: New Opportunities for Connected Data*, 2nd ed. O’Reilly Media, Inc., 2015.
- [4] S. Yao et al., “React: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [5] V. A. Traag, L. Waltman, and N. J. van Eck, “From louvain to leiden: Guaranteeing well-connected communities,” *Scientific Reports*, vol. 9, no. 1, p. 5233, 2019.
- [6] Neo4j, Inc., *Neo4j graph database platform*, 2023. Accessed: May 1, 2025. [Online]. Available: <https://neo4j.com/>
- [7] S. Ramírez, *Fastapi: High performance, easy to learn, fast to code, ready for production*, 2018. Accessed: May 1, 2025. [Online]. Available: <https://github.com/tiangolo/fastapi>
- [8] Ollama Team, *Ollama: Run large language models locally*, 2023. Accessed: May 1, 2025. [Online]. Available: <https://ollama.com/>
- [9] Microsoft Research, *Graphrag: Unlocking llm discovery on narrative private data*, 2024. Accessed: May 1, 2025. [Online]. Available: <https://microsoft.github.io/graphrag/>
- [10] S. Scott-Hayward, G. O’Callaghan, and S. Sezer, “Sdn security: A survey,” in *IEEE SDN for Future Networks and Services*, 2013.
- [11] K. Ashton, *That internet of things thing*, RFID Journal, 2009.
- [12] K. Singhal et al., “Large language models encode clinical knowledge,” *Nature*, vol. 620, no. 7972, pp. 172–180, 2023.
- [13] Z. Liu et al., “Benchmarking retrieval-augmented generation for medicine,” *arXiv preprint arXiv:2402.13178*, 2024.
- [14] S. Es, J. James, A. Sivaraman, B. Karthik, Y. Du, et al., “Ragas: Automated evaluation of retrieval augmented generation,” *arXiv preprint arXiv:2309.15217*, 2023.
- [15] P. Chandak, K. Huang, and M. Zitnik, “Building a precision medicine knowledge graph for drug repurposing and clinical decision support,” *Scientific Data*, vol. 10, no. 1, p. 308, 2023.

- [16] J. Wang, X. Yi, R. Guo, et al., “Milvus: A purpose-built vector data management system,” in *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*, 2021, pp. 2614–2627.
- [17] M. Jones, J. Bradley, and N. Sakimura, *Json web token (jwt)*, RFC 7519, 2015.
- [18] H. Chase, *Langchain*, 2022. [Online]. Available: <https://github.com/langchain-ai/langchain>
- [19] S. Jin, Y. Zhang, Y. Wang, et al., “A survey on biomedical named entity recognition,” *Journal of Biomedical Informatics*, vol. 125, p. 103975, 2022.
- [20] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “Llm.int8(): 8-bit matrix multiplication for transformers at scale,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 22183–22196, 2022.
- [21] N. Francis et al., “Cypher: An evolving query language for property graphs,” in *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, 2018, pp. 1433–1445.
- [22] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, “Fast unfolding of communities in large networks,” *Journal of statistical mechanics: theory and experiment*, vol. 2008, no. 10, P10008, 2008.
- [23] C.-Y. Lin, “Rouge: A package for automatic evaluation of summaries,” in *Text Summarization Branches Out*, 2004, pp. 74–81.
- [24] K. Papineni, S. Roukos, T. C. Ward, and W.-J. Zhu, “Bleu: A method for automatic evaluation of machine translation,” in *Proceedings of the 40th annual meeting of the Association for Computational Linguistics (ACL)*, 2002, pp. 311–318.
- [25] A. Vaswani et al., “Attention is all you need,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [26] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” *arXiv preprint arXiv:1810.04805*, 2018.
- [27] Qwen Team, “Qwen2.5 technical report,” *arXiv preprint arXiv:2412.15115*, 2024.

APPENDIX

A. USE CASE DESCRIPTIONS

This appendix provides detailed use case specifications for the core functions of the Smart Medical QA & Consultation System.

A.1 Use Case UC-01: Consult Medical Agent

Use Case UC-01 – Consult Medical Agent: Actors include clinicians, patients, or administrative staff. The user queries the intelligent agent in natural language, triggering a localized ReAct loop that searches the custom Neo4j Knowledge Graph, retrieves clinical evidence chunks, inspects pill details, and constructs a structured Vietnamese response with Vis.js network citations. Preconditions are that the local Neo4j service is running with a KG of 12,942 entities and the local backend hosts the 3-billion parameter model. In the normal flow, the user enters a clinical question (e.g., "Diabetes diet suggestions"), the system detects intent and invokes the ReAct pipeline, the agent generates a thought and runs the `graphrag_query` tool, Cypher queries retrieve matching entities and context paragraphs, the agent synthesizes evidence in a second iteration and outputs the `Final Answer`, and the Web UI streams markdown, renders a Vis.js subgraph, and offers PDF export. Alternative flows include format recovery (repairing missing ReAct tags), loop-guard trigger (terminating duplicate tool calls), and intent routing fallback (direct quantitative lab checks).

A.2 Use Case UC-02: Import and Analyze Multi-format Lab Report

Use Case UC-02 – Import and Analyze Multi-format Lab Report: Actors are clinicians or patients. The user uploads a digital lab report in PDF or Excel format; the system extracts textual metrics, automatically compares them against on-form reference ranges, highlights abnormal parameters, and generates a personalized clinical consultation using graph-derived recommendations. The precondition is a valid local file containing standard laboratory grids. In the normal flow, the user drops the file onto the Web UI, the FastAPI server streams it to the PyMuPDF or openpyxl extractor, the extracted grids are processed by the `lab_compare_on_form` module, the system flags indicators as normal, high, or low, extracts abnormal entities, queries Neo4j for recommendations, prompts the LLM for an advice plan, and the UI displays side-by-side color-coded alerts and advisory text.

A.3 Use Case UC-03: Manage Medication Schedule and Reminder

Use Case UC-03 – Manage Medication Schedule and Reminder: Actors are patients or caregivers. The user sets up daily intake schedules for suggested medications, visualizes schedule blocks on a weekly calendar, and receives automated audio-visual reminders. The precondition is that the agent has generated a prescription plan or the user manually inputs drug names. In the normal flow, the user opens the Reminder Calendar, selects a medication card, defines frequency, target times, and intake markers, the system stores the configuration in browser local storage,

the Web UI updates the weekly schedule with colored blocks, and the browser triggers an alert badge and audio tone when a time slot is reached.

A.4 Use Case UC-04: Clinical Calculators and Interaction Safety Check

Use Case UC-04 – Clinical Calculators and Interaction Safety Check: Actors are clinicians or patients. The user employs deterministic tools to calculate standard metrics (BMI, eGFR) or queries the Neo4j Knowledge Graph to check drug-drug interaction safety. Preconditions are that the user has accessed the "Công cụ" page and the backend API is connected to Neo4j. In the normal flow, the user navigates to the clinical tools interface, inputs weight and height to calculate BMI (instant result with classification), inputs serum creatinine, age, and sex to compute eGFR using the CKD-EPI (2021) equation, or selects two drug names to check interaction safety, where the backend queries Neo4j and returns clinical warning text and source evidence.

B. CORE SYSTEM PROMPTS

This appendix contains the core system prompts used to configure the behavior of the Large Language Model (LLM) agents within the system, particularly the ReAct Agent.

ReAct Agent System Prompt

Bạn là trợ lý thông tin y tế (tiếng Việt). Tránh chẩn đoán dứt khoát khi thiếu căn cứ; có thể đề xuất thuốc, liều và cách dùng phù hợp khi câu hỏi cần.

Bạn có các công cụ:

graphrag_query: Tra cứu kho tri thức y khoa đồ thị (GraphRAG). Dùng khi cần căn cứ tài liệu đã chỉ mục: triệu chứng, thuốc, bệnh, xét nghiệm, phòng bệnh, v.v. Action Input: ưu tiên copy nguyên văn câu hỏi trong Question (tiếng Việt). Chỉ thêm từ khóa tối thiểu nếu cần; tránh đổi sang diễn đạt khác kiểu «tương tác giữa...» vì kho QA khớp fulltext theo câu gốc tốt hơn.

pill_image_lookup: Tìm ảnh minh họa thuốc trong dataset đã crawl (local). Dùng khi người dùng hỏi hình dạng/màu sắc/ảnh viên thuốc, hoặc muốn xem ảnh tham khảo. Action Input: tên hoạt chất / từ khóa tiếng Anh khớp thư mục crawl (vd. acetaminophen, ibuprofen), ngắn gọn.

medical_calculator: Tính toán các chỉ số y khoa lâm sàng chuẩn như chỉ số khối cơ thể (BMI) hoặc độ lọc cầu thận (eGFR) để đánh giá chức năng thận của bệnh nhân. Dùng khi người dùng yêu cầu tính toán và đã cung cấp các thông số như chiều cao, cân nặng, độ tuổi, giới tính hoặc chỉ số Creatinine máu. Action Input: Một chuỗi JSON hợp lệ.

drug_interaction_checker: Tra cứu tương tác và chống chỉ định đồng thời giữa hai hoặc nhiều loại thuốc trực tiếp từ đồ thị tri thức Neo4j. Dùng khi người dùng hỏi về việc dùng chung các loại thuốc. Action Input: Một chuỗi JSON hợp lệ chứa tên các thuốc cần kiểm tra.

Quy trình ReAct – mỗi lượt trả lời một trong hai định dạng sau (không để trống Thought):

A) Cần sử dụng công cụ:

Thought: (rất ngắn, 1-2 câu) vì sao cần sử dụng công cụ.

Action: graphrag_query hoặc pill_image_lookup hoặc medical_calculator hoặc drug_interaction_checker

Action Input: <chuỗi tìm kiếm hoặc JSON phù hợp tool>

Observation:

B) Kết thúc trả lời:

Thought: (rất ngắn, 1-2 câu) lý do kết thúc.

Final Answer: <trả lời trực tiếp cho người dùng>

ReAct Parse Retry Prompt

Định dạng không hợp lệ. [error_message]

Hãy trả lời lại một trong hai: (1) Action: graphrag_query hoặc pill_image_lookup hoặc medical_calculator + Action Input: ... + dòng Observation: trống hoặc (2) Final Answer: ...

Không bọc ** quanh nhãn Thought/Action. Câu hỏi dài: copy nguyên Question vào một dòng Action Input.

C. DEPLOYMENT GUIDE

This appendix outlines the steps required to deploy the system locally using Docker and Python.

1. Clone the Repository

Start by cloning the source code repository to your local machine and navigating into the project directory:

```
git clone <REPOSITORY_URL>
cd DATN
```

2. System Prerequisites

To successfully deploy this system, several dependencies must be met: (i) Docker and Docker Compose must be installed for the database, (ii) Python 3.10 or higher is required for the backend services, and (iii) Ollama must be installed locally for LLM inference.

3. Python Environment Setup

It is highly recommended to isolate dependencies using a Python virtual environment. Initialize and activate the environment, then install the required packages:

```
python -m venv .venv
# On Windows (PowerShell):
.\.venv\Scripts\Activate.ps1
# On Linux/macOS:
source .venv/bin/activate

pip install --upgrade pip
pip install -r requirements.txt
```

4. Environment Variables Configuration

Create a local environment file from the provided template:

```
cp config/.env.example config/.env
```

Modify the `config/.env` file if necessary (e.g., setting the Neo4j authentication credentials or LLM model configurations).

5. Infrastructure Setup (Neo4j)

Start the Neo4j database using Docker Compose in detached mode. This provisions the database and required APOC plugins automatically:

```
docker-compose up -d
```

6. Knowledge Graph Ingestion

There are two alternative methods to populate the Neo4j Graph Database:

Method A: Fast-Track Ingestion using Pre-extracted Dataset (Recommended)

To skip the computationally expensive LLM extraction phase, you can ingest the pre-extracted clinical knowledge graph directly:

1. Download the pre-extracted dataset files (*.jsonl) from Kaggle at:
`https://www.kaggle.com/datasets/binhduongvuongle/knowledge-graph-data-about-medical-consultant`
2. Unzip and place all the downloaded files directly into the project directory at `kg/kg_artifacts/`.
3. Run the schema and import scripts to populate Neo4j in less than a minute:

```
python scripts/kg_apply_schema.py
python scripts/kg_import_artifacts.py
```

Method B: Raw Extraction and Ingestion from Scratch

To extract clinical entities and relations from raw corpora using local LLMs:

1. Download the raw datasets from Hugging Face:

```
python scripts/download_additional_medical_data.py
python scripts/download_vietnamese_medical_qa.py
python scripts/download_hf_dataset.py
```

1. Clean the data and start the incremental extraction and ingestion process:

```
python scripts/clean_vi_medical_data.py
python scripts/clean_vi_qa_data.py
python scripts/kg_apply_schema.py
python scripts/kg_extract_entities.py --only-missing
python scripts/kg_extract_relations.py --only-missing
```

7. Start Ollama and Pull Models

Ensure Ollama is running and pull the necessary local models for offline inference:

```
ollama serve
ollama pull llama3.2:3b
ollama pull qwen2.5:1.5b-instruct
```

8. Start the FastAPI Backend

Run the backend application server on the local network:

```
python -m uvicorn api.main:app --host 127.0.0.1 --port 8000 --reload
```

The system Web UI is now accessible via the endpoints hosted on <http://127.0.0.1:8000/ui/>.

D. DATABASE DICTIONARY

This appendix provides the Data Dictionary for the Neo4j Knowledge Graph schema.

Nodes

Label	Property	Description
Entity	entity_id	Unique string identifier (e.g., 'DISEASE_DIABETES')
	canonical_name	Standardized medical name (e.g., 'Diabetes Mellitus')
	type	Biomedical type (e.g., Disease, Drug, Symptom)
	aliases	Array of string synonyms
Chunk	chunk_id	Unique hash identifier for the text chunk
	text	The raw clinical text
	document_id	Foreign key to the parent Document
Document	doc_id	Unique identifier for the source document
	title	Document title or URL
	metadata	JSON string of additional attributes

Table D.1: Node schema in Neo4j Knowledge Graph

Relationships

Type	Nodes	Description
REL	(Entity) → (Entity)	Logical biomedical relationship (e.g., treats, causes). Contains property 'predicate' and 'confidence'.
MENTIONS	(Chunk) → (Entity)	Links a text chunk to an extracted medical entity.
HAS_CHUNK	(Document) → (Chunk)	Links a document to its constituent chunks.

Table D.2: Relationship schema in Neo4j Knowledge Graph

E. CORE CODE SNIPPETS

This appendix provides core code snippets demonstrating the system's runtime behavior.

ReAct Format Healer (Regex Parser)

```
def extract_fallback_answer(self, text: str) -> str | None:
    """TrytoextractsomeanswerifthestRICTReActformatcompletelyfailed."""
    ""
    import re
    # Try to find anything after Final Answer regardless of
    formatting
    match = re.search(r"Final\s*Answer\s*:\s*(.*)", text, re.
    IGNORECASE | re.DOTALL)
    if match:
        ans = match.group(1).strip()
        ans = re.sub(r"^(``|['\"])+|['\"]+$", "", ans)
        if ans:
            return ans

    # Try to find a standalone observation-less response
    if "graphrag_query" not in text and len(text) > 50:
        clean = re.sub(r"\^+", "", text).strip()
        if not clean.startswith("Thought:") and not clean.startswith(
        "Action:"):
            return clean

    return None
```

GraphRAG Retrieval Context Injection

```
def _append_answer_source_note(answer: str, graph_context: str,
    retrieval_hits: list[dict[str, Any]]) -> str:
    """Appendsource/methodnoteunderanswerforUItransparency."""
    base = (answer or "").strip()
    marker = "Nguontraloi:"
    if marker in base:
        return base

    if (graph_context or "").strip() or retrieval_hits:
        source_note = "RAG (GraphRAG+LLMtonghop) "
    else:
        source_note = "LLMtructiep(khongdungRAG) "
```

```
if not base:
    return f"{marker}{source_note}"
return f"{base}\n\n---\n{marker}{source_note}"
```